

7장 서비스 2

7.1 LoadBalancer 서비스

7.1.1 클라우드환경 LoadBalancer 서비스 >> AWS CLB/NLB

7.1.2 온프레미스환경 LoadBalancer 서비스 >> MetalLB 소개

7.1.3 실습 환경 구성

7.2 MetalLB - Layer2 모드

7.2.1 Layer2 모드 소개

7.2.2 MetalLB 설치 - Layer2 모드

7.2.3 파드 생성

7.2.4 서비스 생성 (MetalLB/ARP 동작 확인)

7.2.5 서비스 접속

7.2.6 Failover 테스트

7.3 MetalLB - BGP 모드

7.3.1 BGP 모드 소개

7.3.2 MetalLB 설치 - BGP 모드

7.3.3 파드 & 서비스 생성 (MetalLB/BGP 동작 확인)

7.3.4 서비스 접속

7.3.5 externalTrafficPolicy 설정

7.3.6 Failover 테스트

7.4 CoreDNS

7.4.1 CoreDNS 소개

7.4.2 파드 DNS

7.4.3 서비스 DNS

7.4.4 ExternalName 서비스

7.4.5 스테이트풀셋 & 헤드리스서비스

7.5 IPVS Proxy 모드

7.5.1 실습 환경 구성 / 7.5.2 Cluster 서비스

7.5.3 NodePort 서비스 / 7.5.4 LoadBalancer 서비스

7.1 LoadBalancer 서비스 소개

서비스<Service>의 종류 중 하나인 LoadBalancer 서비스에 대해서 알아보겠습니다.

LoadBalancer 서비스는 **퍼블릭 클라우드 환경**에서 동작하는 LoadBalancer 서비스와 **온프레미스** <on-premises> **환경**에서 동작하는 LoadBalancer 서비스가 있습니다.

동일한 LoadBalancer 서비스이지만, 동작하는 방식이 차이가 있습니다.

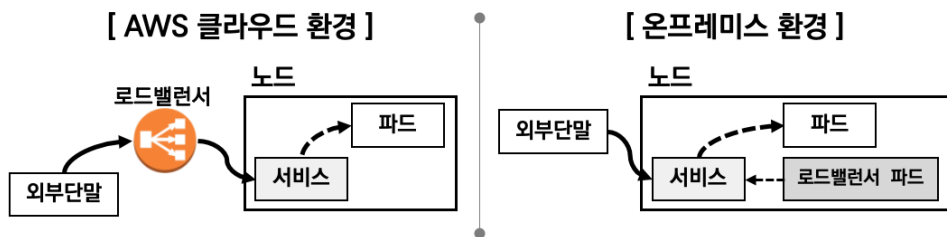
퍼블릭 클라우드 사업자마다 LoadBalancer 서비스의 동작에도 차이가 조금씩 있습니다.

마찬가지로 온프레미스 환경에서 동작하는 LoadBalancer 서비스도 하드웨어 장비 기반과 소프트웨어 기반으로 구분되어 지며, 심지어 소프트웨어 LoadBalancer 서비스도 개발사에 따라 동작하는 방식에 차이가 있습니다.

해당 책에서는 가장 많이 사용되는 LoadBalancer 서비스를 기준하여 설명합니다.

퍼블릭 클라우드 환경은 **AWS** LoadBalancer 서비스를 소개하며, 온프레미스 환경은 **MetalLB** 소프트웨어 LoadBalancer 서비스를 소개합니다.

[그림 7-1] LoadBalancer 서비스 비교



AWS 클라우드 환경의 AWS LoadBalancer 서비스는 이론 소개만 진행하고 실습 내용은 별도로 다루지 않습니다. 참고로 저희 팀에서 추후에 '*가칭* AWS EKS 인프라 입문' 책을 집필 할때 AWS LoadBalancer 에 대해서 자세히 다룰 예정입니다.

7.1.1 클라우드 환경 LoadBalancer 서비스

AWS 클라우드 환경의 AWS LoadBalancer 서비스에 대해서 알아보겠습니다.

AWS 는 AWS 관리형 LoadBalancer 서비스인 **ELB**< Elastic Load Balancing> 가 있습니다.

AWS ELB 에는 3가지 종류인 CLB, NLB, ALB 가 있습니다.

CLB<Classic Load Balancing > 는 클래식 로드밸런서입니다. NLB/ALB에 비해서 가장 적은 기능을 제공하며 가장 오래된 로드밸런서입니다.

NLB<Network Load Balancing > 는 네트워크 로드밸런서입니다. CLB/ALB에 비해서 처리 속도가 가장 빠르고 Layer4 계층에서 동작합니다. TCP/UDP/TLS 트래픽을 처리할 수 있습니다.

ALB<Application Load Balancing> 는 애플리케이션 로드밸런서입니다. HTTP/HTTPS/gPRC 트래픽을 전문으로 처리하며, Layer7 계층에서 동작합니다.

쿠버네티스에서 **LoadBalancer** 서비스 리소스를 생성 시 AWS CLB 나 AWS NLB 가 생성되어 노드에 있는 파드로 부하분산 연결됩니다.

AWS ALB 는 쿠버네티스에서 **Ingress** 리소스를 생성 시 사용됩니다. Ingress 리소스는 8장에서 상세히 다룰 예정입니다. 따라서 이번 장에서는 AWS ALB 대신 Nginx Ingress 를 이용한 실습으로 알아볼 예정입니다.

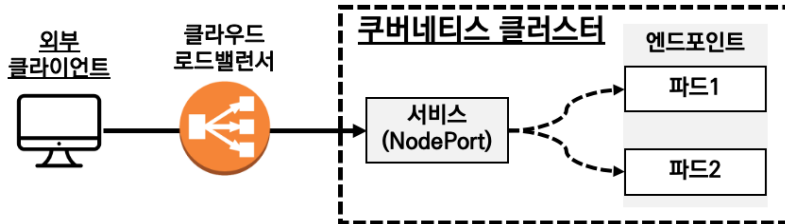
그럼 쿠버네티스에서 LoadBalancer 서비스 리소스를 생성 시 AWS CLB 나 AWS NLB 는 어떻게 파드와 통신되는지 알아보겠습니다.

크게 2가지 동작 방식이 있습니다.

첫 번째 방식은 외부 클라이언트는 로드밸런서로 접속을 하고, 로드밸런서는 파드가 있는 노드들로 부하분산해서 전달합니다. 이때 노드의 NodePort 를 목적지 포트로 전달하게 됩니다. 노드의 NodePort 로 인입 후에는 iptables 에 의해서 파드로 랜

덤 부하분산을 통해서 전달합니다. 즉 해당 경우에는 부하분산 과정이 두 번 수행이 됩니다.

[그림 7-2] AWS LoadBalancer 서비스 동작 방식 1



두 번째 방식은 로드밸런서에서 파드의 IP로 직접 부하분산 전달합니다. 로드밸런서가 파드의 IP 정보를 알기 위해서, 별도의 로드밸런서 컨트롤러를 구성하고 해당 로드밸런서 컨트롤러가 로드밸런서에게 파드의 IP를 동적으로 전달합니다. 부하분산 과정을 한 번만 수행하여 첫 번째 방식에 비해서 좀 더 효율적입니다. 참고로 해당 동작을 위해서는 AWS 로드밸런서 컨트롤러를 구성해야 합니다.

[그림 7-3] AWS LoadBalancer 서비스 동작 방식 2



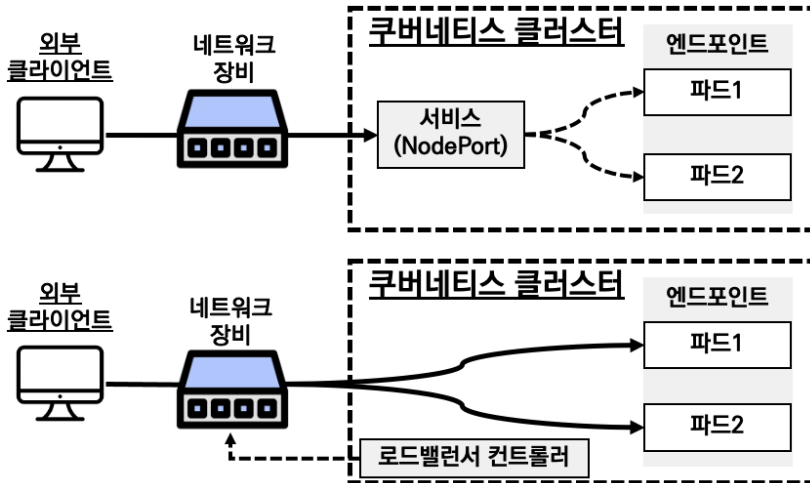
7.1.2 온프레미스 환경 LoadBalancer 서비스

온프레미스 환경의 LoadBalancer 서비스에 대해서 알아보겠습니다.

마찬가지로 온프레미스 환경에서 동작하는 LoadBalancer 서비스도 하드웨어 장비 기반의 동작을 먼저 알아보겠습니다.

클라우드 환경의 AWS LoadBalancer 서비스에 거의 동일하게 별도의 장비로 접속 후 노드에 NodePort 혹은 파드로 직접 전달하여 통신 할 수 있습니다.
대표적으로 시트릭스나<citrix> 나 F5 네트워크의 제품이 있습니다.

[그림 7-4] 하드웨어 장비 기반 LoadBalancer 서비스 동작 방식



온프레미스 환경에서 동작하는 LoadBalancer 서비스도 소프트웨어 기반의 동작은 별도의 네트워크 장비 없이, 소프트웨어로 동작합니다.

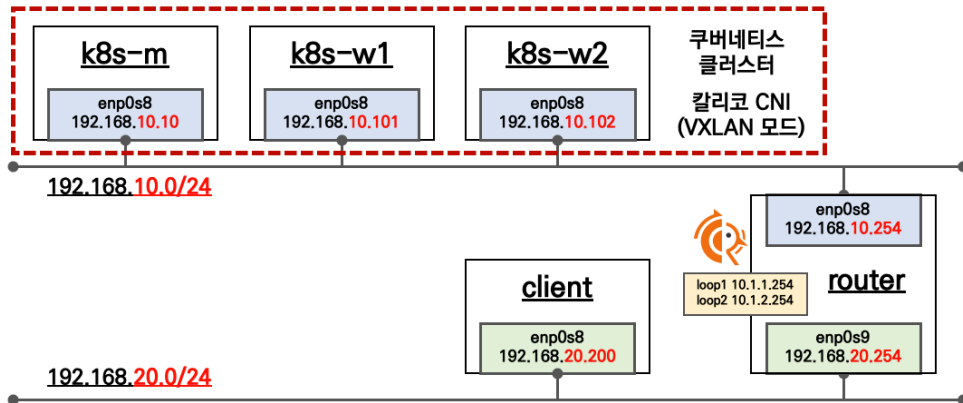
대표적으로 MetalLB , OpenELB, PubeLB, kube-vip 등이 있습니다. 소프트웨어 기반의 LoadBalancer 서비스 동작은 거의 유사합니다.

그럼 MetalLB 를 직접 설치 후 동작에 대해서 상세히 알아보도록 하겠습니다.

7.1.3 실습 환경 구성

칼리코 CNI 를 사용하며, 칼리코의 VXLAN 모드로 파드 간 통신을 합니다.

[그림 7-5] 실습 환경 구성도



vagrant 로 5대의 가상머신을 배포하며, 쿠버네티스 클러스터에 마스터 노드 1대와, 워커 노드 2대(k8s-w1 k8s-w2) 로 구성됩니다. router 이름의 가상머신은 2개의 네트워크 인터페이스를 가지며, 192.168.10.0 대역과 192.168.20.0 대역간 통신을 위한 라우팅 기능을 수행합니다. client 이름의 가상머신은 서비스 접속 테스트 용도로 사용됩니다.

실습 환경을 배포하겠습니다.

```
터미널
# 베이그런트 스크립트 파일을 다운로드
curl -O https://raw.githubusercontent.com/gasida/book-k8s-network/main/7/Vagrantfile

# 배포 실행 : PC 환경에 따라 다르지만, 보통 20분~30분 정도 소요됨
vagrant up

# 배포된 가상머신 상태 확인 : 5대의 가상머신이 정상 동작 중
vagrant status

Current machine states:
k8s-m                running (virtualbox)
router               running (virtualbox)
```

```
k8s-w1      running (virtualbox)
k8s-w2      running (virtualbox)
client      running (virtualbox)
```

SSH 접속 : 각각 가상머신 접속

```
vagrant ssh k8s-m
```

```
vagrant ssh router
```

```
vagrant ssh k8s-w1
```

```
vagrant ssh k8s-w2
```

```
vagrant ssh client
```

설치 후 환경을 간단히 확인해보겠습니다.

터미널

SSH 접속 : 마스터 노드 접속

```
vagrant ssh k8s-m
```

노드 정보 확인 : 워커 노드 3대 확인

```
root@k8s-m:~# kubectl get node
```

NAME	STATUS	ROLES	AGE	VERSION
k8s-m	Ready	control-plane,master	114m	v1.23.6
k8s-w1	Ready	<none>	112m	v1.23.6
k8s-w2	Ready	<none>	110m	v1.23.6

파드와 서비스가 사용 할 수 있는 네트워크 대역 정보 확인

```
root@k8s-m:~# kubectl cluster-info dump | grep -m 2 -E "cluster-cidr|service-cluster-ip-range"
"--service-cluster-ip-range=10.200.1.0/24",
"--cluster-cidr=172.16.0.0/16",
```

칼리코 모드 정보 확인 : VXLAN 모드 사용 중

```
root@k8s-m:~# calicoctl get ippool -owide
```

NAME	CIDR	NAT	IPIP MODE	VXLAN MODE	DISABLED	DISABLE BGP EXPORT	SELECTOR
default-ipv4-ippool	172.16.0.0/16	true	Never	Always	false	false	all()

칼리코 VXLAN 네트워크 인터페이스 확인 : VXLAN 네트워크 오버레이 통신에 사용되는 인터페이스

```
root@k8s-m:~# ip -c -4 addr show type vxlan
```

```
9: vxlan.calico: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1450 qdisc noqueue state UNKNOWN group default
    inet 172.16.116.0/32 scope global vxlan.calico ...(생략)...
```

7.2 MetalLB Layer2 모드

MetalLB는 BareMetalLoadBalancer의 약자로, 온프레미스 환경에서 표준 프로토콜을 사용해서 LoadBalancer 서비스를 구현해주는 프로그램입니다.

쿠버네티스의 데몬셋<DaemonSet> 으로 스피커<Speaker> 파드를 생성하여, 'External IP'를 전파합니다. External IP 는 노드의 IP 대신 외부에서 접속 할 수 있습니다. 이를 통해 노드의 IP를 외부에 노출하지 않을 수 있어서 보안성을 높일 수 있습니다.

스피커 파드는 External IP 전파를 위해서 표준 프로토콜인 ARP<Address Resolution Protocol> 혹은 BGP 를 사용합니다. MetalLB는 ARP를 사용하는 Layer2 모드와 BGP를 사용하는 BGP 모드 중 하나를 선택하여 동작합니다.

MetalLB는 대부분의 퍼블릭 클라우드 플랫폼 환경에서는 동작하지 않습니다. 이유는 가상서버의 IP에 매칭되는 MAC 주소가 아닌 IP에 대한 ARP 요청을 차단하기 때문입니다. MetalLB 사용 시 External IP는 퍼블릭 클라우드 사업자가 할당한 IP가 아니므로 External IP에 대한 ARP 요청이 차단되어, 결국 MetalLB 동작이 불가능합니다.

또한 일부 CNI는 동작에 이슈가 있습니다.¹ 예를 들면 칼리코의 IPIP 모드에 BGP를 사용하면서 MetalLB BGP 모드를 사용 시 BGP 동작에 충돌로 문제가 발생할 수 있습니다.

실무에서 사용 시에는 이슈나 제약사항을 꼭 확인하고 사전 테스트를 진행 후 사용하기를 권장합니다.

¹ 출처 <https://metallb.universe.tf/installation/network-addons/>

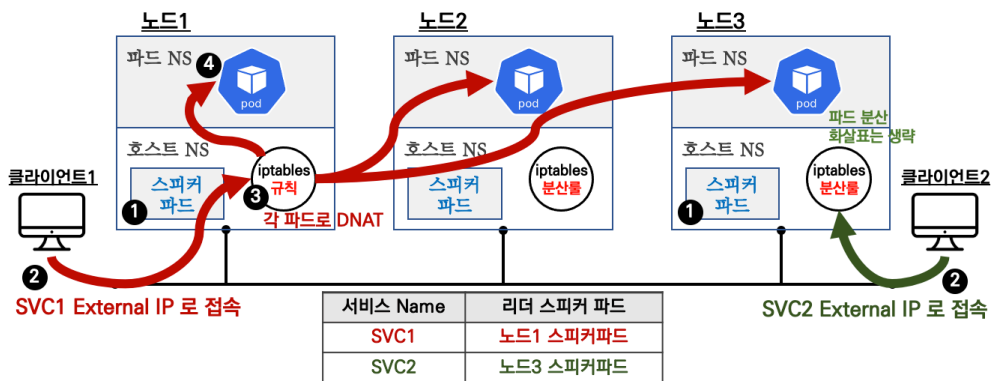
7.2.1 Layer2 모드 소개

MetalLB는 **Layer2 모드** 동작을 알아보겠습니다.

Layer2 모드는 ARP 를 통해서 External IP 를 전파하기 때문에 ARP 동작을 간단히 설명드리겠습니다.

동일 네트워크 내부에서는 통신을 위해서는 상대방 장비의 MAC <media access control> 주소를 알아야합니다. 이때 ARP 프로토콜을 통해서 상대방 IP 주소에 해당하는 MAC 주소를 요청하고 응답을 통해서 자동으로 알아오게 됩니다.

[그림 7-6] MetalLB 는 Layer2 모드 동작



위 그림에 '호스트 NS/파드 NS'에서 NS는 네임스페이스 <Namespace> 의 약자입니다.

(1) LoadBalancer 서비스 리소스 생성 시 MetalLB 스피커파드들 중 리더 <Leader> 스피커파드가 선출됩니다. 리더 스피커파드는 해당 LoadBalancer 서비스의 External IP 를 전파합니다. Layer2 모드이므로 ARP(GARP, Gratuitous ARP) 메시지로 동일 네트워크 대역 내에 브로드캐스트 <broadcast> 주소로 전파합니다.

만약 리더 스피커파드가 장애가 발생하게 되면, 나머지 스피커파드 중에서 다시 리더 스피커파드가 선출됩니다.

위 그림에서는 SVC1 서비스는 노드1에 있는 스피커파드가 리더이며, SVC2 서비스는 노드3에 있는 스피커파드가 리더입니다. 각각 자신의 LoadBalancer 서비스의 External IP 를 ARP 를 통해서 전파를 하고 있는 상태입니다.

(2) 클라이언트1은 SVC1 External IP로 접속을 시도합니다. 해당 트래픽은 SVC1 External IP 정보를 전파하는 리더 스피커파드가 있는 노드1로 들어오게 됩니다. 마찬가지로 클라이언트2은 SVC2 External IP로 접속을 시도하고, 해당 트래픽은 노드3으로 들어오게 됩니다.

(3) 노드에 인입 후 IPTABLES 규칙에 의해서 ClusterIP 와 동일하게 해당 서비스에 연동된 엔드포인트 파드들로 (4) 랜덤 부하분산 되어 전달됩니다.

7.2.2 MetalLB 설치

마스터노드에 접속하여 MetalLB 설치를 진행하겠습니다.

공식 홈페이지에 제공하는 YAML 파일을 사용하며, Layer 2 모드 설정을 하겠습니다.

터미널

```
# MetalLB 관련 파드가 사용할 네임스페이스를 생성
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/namespace.yaml

# 네임스페이스 생성 확인
root@k8s-m:~# kubectl get namespaces metallb-system
NAME          STATUS  AGE
metallb-system Active  56s

# MetalLB 설치 : 디플로이먼트와 데몬셋 등 다양한 리소스가 설치됨
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/metallb.yaml

# MetalLB 리소스 설치 확인 : 해당 YAML 파일로 설치하는 리소스의 종류와 상태를 확인 가능
kubectl get -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/metallb.yaml
...(생략)...

# MetalLB 컨트롤러 파드 확인 : MetalLB 컨트롤러는 클러스터 범위에서 IP를 관리
root@k8s-m:~# kubectl get pod -n metallb-system -l component=controller
```

```

NAME          READY STATUS  RESTARTS  AGE
controller-57fd9c5bb-hlw17  1/1   Running  0         5m46s

# MetalLB 스피커 파드 확인 : 데몬셋으로 배포되어 모든 노드에 MetalLB 스피커 파드가 배포됨
# 파드 IP는 호스트 모드를 사용하여, 배포된 노드의 IP를 그대로 사용함
root@k8s-m:~# kubectl get pod -n metallb-system -l component=speaker -owide
NAME          READY STATUS  RESTARTS  AGE  IP            NODE    NOMINATED NODE  READINESS
speaker-64fm4  1/1   Running  0         8m29s  192.168.10.102 k8s-w2  <none>          <none>
speaker-7dhn1  1/1   Running  0         8m29s  192.168.10.101 k8s-w1  <none>          <none>
speaker-gks85  1/1   Running  0         8m29s  192.168.10.10  k8s-m   <none>          <none>

# MetalLB Layer 2 모드를 위한 컨피그맵 확인
root@k8s-m:~# cat ~/book-k8s-network/7/1-metallb-layer2.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: metallb-system # 해당 컨피그맵은 metallb-system 네임스페이스에 속함
  name: config
data:
  config: |
    address-pools:
    - name: default
      protocol: layer2 # MetalLB Layer2 모드로 동작
      addresses:
      - 192.168.10.201-192.168.10.210 # LoadBalancer 서비스 생성 시 MetalLB 가 할당할 External IP 주소 대역 지정

# MetalLB Layer 2 모드를 위한 컨피그맵 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/1-metallb-layer2.yaml

# 컨피그맵 생성 확인
root@k8s-m:~# kubectl get cm config -n metallb-system
NAME  DATA  AGE
config 1      22s

```

7.2.3 파드 생성

서비스의 엔드포인트인 파드를 생성하겠습니다.

6장에서 사용했던 간단한 웹 서버의 파드이며, 워커노드1과 노드2에 배포하겠습니다.

터미널

```
# 파드 생성을 위한 yaml 파일 확인
root@k8s-m:~# cat ~/book-k8s-network/7/2-webpod.yaml
...(생략)...

# 파드 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/2-webpod.yaml

# 파드 생성 확인 : 파드의 IP는 실습 환경에 따라 다를 수 있음
root@k8s-m:~# calicoctl get wep

WORKLOAD  NODE    NETWORKS      INTERFACE
webpod1   k8s-w1  172.16.158.2/32  cali778ba511bf9
webpod2   k8s-w2  172.16.184.2/32  cali798bceab6f5

# webpod 파드의 IP를 변수에 지정
root@k8s-m:~# WEBPOD1=$(kubectl get pod webpod1 -o jsonpath={.status.podIP})
root@k8s-m:~# WEBPOD2=$(kubectl get pod webpod2 -o jsonpath={.status.podIP})

# 마스터노드에서 webpod1 로 curl 접속 : 정상 접속 확인
root@k8s-m:~# curl -s $WEBPOD1 | grep Hostname
Hostname: webpod1

# 마스터노드에서 webpod2 로 curl 접속 : 정상 접속 확인
root@k8s-m:~# curl -s $WEBPOD2 | grep Hostname
Hostname: webpod2
```

7.2.4 서비스 생성

LoadBalancer 타입의 서비스 3개를 생성해보겠습니다.

터미널

```
# LoadBalancer 타입의 서비스 생성을 위한 yaml 파일 확인
root@k8s-m:~# cat ~/book-k8s-network/7/3-svc1b.yaml

apiVersion: v1
kind: Service
metadata:
  name: svc1
spec:
  ports:
    - name: svc1-webport
      port: 80
      targetPort: 80
  selector:
    app: webpod
  type: LoadBalancer # 서비스 타입이 LoadBalancer 임
---
...(생략)...
```

```
# LoadBalancer 타입의 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/3-svc1b.yaml
```

```
# LoadBalancer 타입의 서비스 생성 확인 : EXTERNAL-IP가 서비스 마다 할당되며, 실습 환경에 따라 다를 수 있음
# LoadBalancer 타입의 서비스는 NodePort 와 ClusterIP 를 포함함
# ExternalIP 로 접속 시 사용하는 포트는 PORT(S) 의 앞에 있는 값을 사용 (아래의 경우는 TCP 80 임)
# 만약 노드의 IP에 NodePort 로 접속 시 사용하는 포트는 PORT(S) 의 뒤에 있는 값을 사용 (아래는 32127 임)
root@k8s-m:~# kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
svc1	LoadBalancer	10.200.1.33	192.168.10.201	80:32127/TCP	7m15s
svc2	LoadBalancer	10.200.1.141	192.168.10.202	80:30328/TCP	7m15s
svc3	LoadBalancer	10.200.1.144	192.168.10.203	80:32055/TCP	7m15s

```
...(생략)...
```

```
# svc1 서비스 상세 확인 : 리더 스피커파드 선정은 실습 환경에 따라 다를 수 있음
# ExternalIP 192.168.10.201에 대한 리더 스피커파드는 'k8s-w1' 노드에 있는 스피커파드임
root@k8s-m:~# kubectl describe svc svc1 | grep Events -A5
```

```
Events:
  Type    Reason          Age   From                Message
  ----    -
Normal    IPAllocated     11m   metallb-controller  Assigned IP ["192.168.10.201"]
Normal    nodeAssigned    11m   metallb-speaker     announcing from node "k8s-w1"

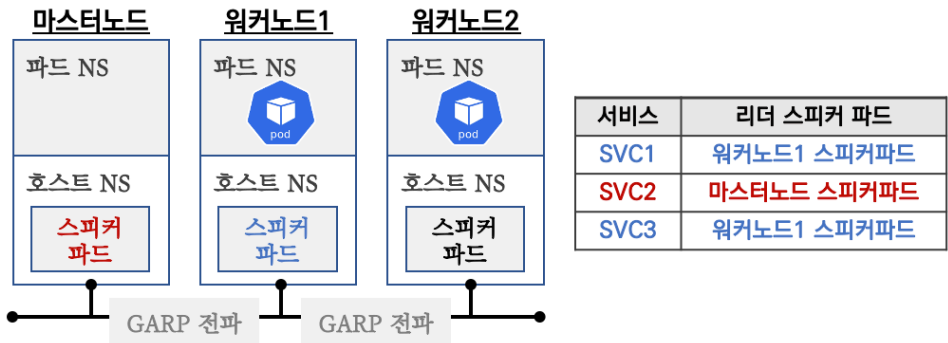
# svc2 서비스 상세 확인 : 리더 스피커파드 선정은 실습 환경에 따라 다를 수 있음
# ExternalIP 192.168.10.202에 대한 리더 스피커파드는 'k8s-m' 노드에 있는 스피커파드임
root@k8s-m:~# kubectl describe svc svc2 | grep Events -A5

Events:
  Type    Reason          Age   From                Message
  ----    -
Normal    IPAllocated     16m   metallb-controller  Assigned IP ["192.168.10.202"]
Normal    nodeAssigned    16m   metallb-speaker     announcing from node "k8s-m"

# svc3 서비스 상세 확인
root@k8s-m:~# kubectl describe svc svc3 | grep Events -A5
...(생략)...
```

LoadBalancer 서비스의 External IP 에 대한 리더 스피커파드가 배치된 노드를 router 가상머신에서 확인해보겠습니다.

[그림 7-7] 서비스와 스피커 파드



LoadBalancer 서비스의 생성 시 1대의 스피커파드가 리더 스피커파드가 선정되고, 해당 스피커파드는 GARP 를 전파하여 ExternalIP 에 대한 정보를 전파합니다. 이 해하기 쉽게 설명하자면, ExternalIP 에 대한 소유자라고 동일 네트워크 대역 내에서

떠드는 것입니다. 위 그림에서 SVC1, SVC3 의 ExternalIP 에 대한 소유자는 워커 노드1에 있는 스피커파드이고, SVC2 에 대한 ExternalIP 에 대한 소유자는 마스터 노드에 있는 스피커파드입니다. 참고로 실습 환경에 따라서 리더 스피커파드가 배치 된 노드는 다를 수 있습니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 router 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널1] SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m

# [터미널2] SSH 접속 : router 가상머신 접속
vagrant ssh router

# [터미널1] 마스터노드에서 ARP 에 대한 tcpdump 실행
# 아래 ARPING 실행 시 ARP 요청과 응답 패킷을 확인 할 수 있음
root@k8s-m:~# tcpdump -i enp0s8 -nn arp
...(생략)...

# [터미널2] 서비스 ExternalIP 를 변수에 지정
root@router:~# SVC1EXIP=$(kubectl get svc svc1 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@router:~# SVC2EXIP=$(kubectl get svc svc2 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@router:~# SVC3EXIP=$(kubectl get svc svc3 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# [터미널2] ARPING 톨로 서비스 ExternalIP 를 목적으로 ARP 요청에 대한 응답 (MAC 주소) 확인
# svc1/svc3 의 ExternalIP에 대한 ARP 응답은 08:00:27:37:7A:BF이며, 해당 MAC 주소는 워커노드1의 enp0s8 MAC 주소임
root@router:~# for IP in $SVC1EXIP $SVC2EXIP $SVC3EXIP ; do arping -I enp0s8 -f -c 1 $IP; done
ARPING 192.168.10.201 from 192.168.10.254 enp0s8
Unicast reply from 192.168.10.201 [08:00:27:37:7A:BF] 1.341ms
Sent 1 probes (1 broadcast(s))
Received 1 response(s)
ARPING 192.168.10.202 from 192.168.10.254 enp0s8
Unicast reply from 192.168.10.202 [08:00:27:F0:8E:03] 0.993ms
Sent 1 probes (1 broadcast(s))
```

```

Received 1 response(s)
ARPING 192.168.10.203 from 192.168.10.254 enp0s8
Unicast reply from 192.168.10.203 [08:00:27:37:7A:BF] 1.151ms
Sent 1 probes (1 broadcast(s))
Received 1 response(s)

# [터미널2] ARPING 톨로 노드의 IP를 목적지로 ARP 요청에 대한 응답 확인
root@router:~# for IP in k8s-m k8s-w1 k8s-w2 ; do arping -I enp0s8 -f -c 1 $IP; done
...(생략)...

# [터미널2] ARP 정보 확인 : ExternalIP 에 대한 ARP 응답하는 노드의 IP를 확인
# 아래의 경우 192.168.10.201, 203 IP 에 대한 MAC 주소를, 192.168.10.101 노드가 응답함을 의미함
root@router:~# ip -c neighbor | grep enp0s8 | sort
192.168.10.10 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
192.168.10.101 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE
192.168.10.102 dev enp0s8 lladdr 08:00:27:94:6a:a0 STALE
192.168.10.201 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE
192.168.10.202 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
192.168.10.203 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE

```

위에서 알아본 것처럼, 실제 LoadBalancer 서비스의 ExternalIP에 대해서 MetalLB 리더 스피커 파드가 선정되고, 해당 파드가 ARP 응답을 처리합니다. 결국 LoadBalancer 서비스의 ExternalIP와 통신을 하기 위해서는, 리더 스피커 파드가 배포된 워커노드로 트래픽이 들어오게 됩니다.

아래 그림처럼 리더 스피커 파드는 LoadBalancer 서비스의 ExternalIP 의 충돌 방지와 주소 갱신을 위해서 GARP 를 전파합니다.

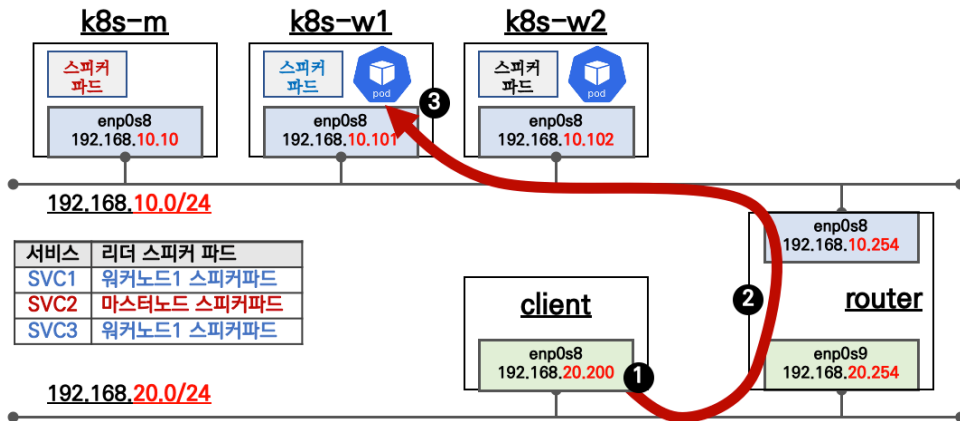
[그림 7-8] 리더 스피커 파드가 전파하는 GARP 패킷 (예시)

Source	Destination	Protocol	Length	Info
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 172.19.0.0 (Request)
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 172.19.0.0 (Reply)
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 192.168.100.200 (Request)
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 192.168.100.200 (Reply)
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 192.168.100.201 (Request)
08:00:27:76:e3:77	ff:ff:ff:ff:ff:ff	ARP	60	Gratuitous ARP for 192.168.100.201 (Reply)

7.2.5 서비스 접속

쿠버네티스 노드가 있는 네트워크 대역이 아닌, 다른 대역에 있는 client 가상머신에서 쿠버네티스 서비스에 접속해보겠습니다.

[그림 7-9] client 가상머신에서 서비스 접속



- (1) client 가상머신에서 SVC1 ExternalIP 와 Port 로 접속을 시도합니다.
- (2) ExternalIP 는 192.168.10.0/24 네트워크 대역이므로, router 가상머신에서 라우팅으로 전달을 하게 됩니다.
- (3) SVC1 ExternalIP 에 대한 리더 스피커파드는 워커노드1에 배치되어 있습니다. 결국 client 요청 트래픽은 워커노드1로 인입됩니다. 워커노드1에 인입 후에는 IPTABLES 규칙에 의해서 서비스에 연동된 파드들로 랜덤 분하분산 됩니다.

실습을 통해서 서비스 접속을 확인해보겠습니다.

[터미널3] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```

터미널
# [터미널3] SSH 접속 : client 가상머신 접속
vagrant ssh client

# [터미널3] 192.168.10.0 대역 라우팅 정보 확인 : 192.168.20.254 는 router 가상머신임
root@client:~# ip -c route | grep 192.168.10
192.168.10.0/24 via 192.168.20.254 dev enp0s8 proto static

```

```
# [터미널3] 서비스 ExternalIP 를 변수에 지정
root@client:~# SVC1EXIP=$(kubectl get svc svc1 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@client:~# SVC2EXIP=$(kubectl get svc svc2 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@client:~# SVC3EXIP=$(kubectl get svc svc3 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# [터미널3] curl 로 서비스 ExternalIP 접속 시도 : 정상적으로 파드에 접속됨을 확인
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC2EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC3EXIP | grep Hostname
Hostname: webpod2

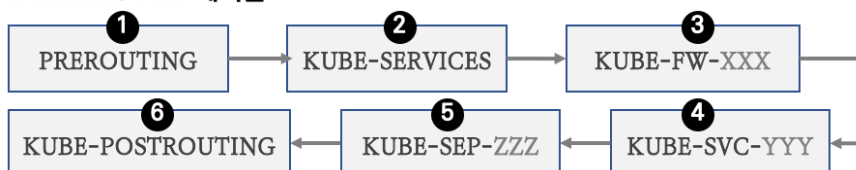
# [터미널3] for 문을 이용하여 서비스1 ExternalIP 로 100번 접속을 시도 : 파드 2개 부하분산 접속 확인
for i in {1..100}; do curl -s --interface 192.168.20.200 $SVC1EXIP | grep Hostname; done | sort | uniq -c | sort -nr
    53 Hostname: webpod1
    47 Hostname: webpod2

# [터미널3] curl 접속 확인 : RemoteAddr 주소는 웹 파드에 인입 패킷의 출발지 주소(IP, Port)가 출력됨
# 접속한 client 의 IP는 192.168.20.200 이지만, 파드에 인입되는 출발지IP는 172.16.158.0 으로 확인됨
# 즉, NodePort 기본 동작과 동일하게 인입한 노드의 인터페이스로 SNAT 되어서 최종 파드로 전달됨
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | egrep 'Hostname|RemoteAddr|Host:'
Hostname: webpod2
RemoteAddr: 172.16.158.0:41874
Host: 192.168.10.201
```

그럼 IPTABLES 규칙을 알아보겠습니다.

[그림 7-10] LoadBalancer 서비스 관련 IPTABLES

IPTABLES NAT 테이블



- (1) 과 (2) 과정은 6장에서 알아본, ClusterIP 와 NodePort 와 동일합니다. KUBE-SERVICES 에서 ExternalIP 마다 KUBE-FW-XXX 체인으로 점프를 하게 됩니다.
- (3) KUBE-FW-XXX 체인 규칙은 KUBE-MARK-MASQ 과 KUBE-SVC-YYY 체인으로 전달합니다. KUBE-MARK-MASQ 체인에서 Mark 를 하고, 추후 (6) 과정에서 클라이언트의 출발지 IP를 SNAT 마스커레이딩 처리를 하게 됩니다.
- (4) KUBE-SVC-YYY 체인 규칙 역시, 6장에서 알아본 내용과 동일합니다. 현재 서비스에 연동된 파드가 2개가 있으므로, (5) 2개의 SEP 규칙이 있고, 랜덤 부하분산 연결하게 됩니다.
- (6) 마지막으로 KUBE-POSTROUTING 규칙에서는 Mark 되어 있는지를 확인하고, Mark 되어 있을 경우 출발지 IP를 SNAT 마스커레이딩 처리를 하게 됩니다.

그럼 간단히 워크노드1에서 IPTABLES 를 확인해보도록 하겠습니다.

[터미널4] 은 워크노드1에 SSH 접속하여 명령어를 실행합니다.

```

터미널
# [터미널4] 서비스1 ExternalIP 를 변수에 지정
root@k8s-w1:~# SVC1EXIP=$(kubectl get svc svc1 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# [터미널4] KUBE-SERVICES 체인의 규칙 확인 : 실습 환경에 따라 체인 규칙 이름을 다를 수 있음
# 서비스1의 ExternalIP TCP 80 매칭되는 트래픽은 KUBE-FW-XXX 체인으로 전달 (-j 점프)
root@k8s-w1:~# iptables -t nat -S KUBE-SERVICES | grep $SVC1EXIP
-A KUBE-SERVICES -d 192.168.10.201/32 -p tcp -m comment --comment "default/svc1:svc1-webport loadbalancer IP" -
m tcp --dport 80 -j KUBE-FW-DLGPAL4ZCYSJ7UPR

# [터미널4] KUBE-FW-XXX 체인의 규칙 확인 : KUBE-MARK-MASQ, KUBE-SVC-DLGPAL4ZCYSJ7UPR 전달 확인
root@k8s-w1:~# iptables -t nat -S KUBE-FW-DLGPAL4ZCYSJ7UPR
-N KUBE-FW-DLGPAL4ZCYSJ7UPR
-A KUBE-FW-DLGPAL4ZCYSJ7UPR -m comment --comment "default/svc1:svc1-webport loadbalancer IP" -j KUBE-
MARK-MASQ
-A KUBE-FW-DLGPAL4ZCYSJ7UPR -m comment --comment "default/svc1:svc1-webport loadbalancer IP" -j KUBE-SVC-
DLGPAL4ZCYSJ7UPR
-A KUBE-FW-DLGPAL4ZCYSJ7UPR -m comment --comment "default/svc1:svc1-webport loadbalancer IP" -j KUBE-
MARK-DROP

```

```
# [터미널4] KUBE-SVC-YYY 체인의 규칙 확인 : KUBE-SEP-ZZZ 규칙 2개(파드 갯수)를 확인, 랜덤 부하분산 됨
root@k8s-w1:~# iptables -t nat -S KUBE-SVC-DLGPAL4ZCYSJ7UPR
-N KUBE-SVC-DLGPAL4ZCYSJ7UPR
-A KUBE-SVC-DLGPAL4ZCYSJ7UPR ! -s 172.16.0.0/16 -d 10.200.1.33/32 -p tcp -m comment --comment "default/svc1:svc1-webport cluster IP" -m tcp --dport 80 -j KUBE-MARK-MASQ
-A KUBE-SVC-DLGPAL4ZCYSJ7UPR -p tcp -m comment --comment "default/svc1:svc1-webport" -m tcp --dport 32127 -j KUBE-MARK-MASQ
-A KUBE-SVC-DLGPAL4ZCYSJ7UPR -m comment --comment "default/svc1:svc1-webport" -m statistic --mode random --probability 0.500000000000 -j KUBE-SEP-EQC4FBRB74NJGSMC
-A KUBE-SVC-DLGPAL4ZCYSJ7UPR -m comment --comment "default/svc1:svc1-webport" -j KUBE-SEP-KPCTGQGMHFC7HTE5

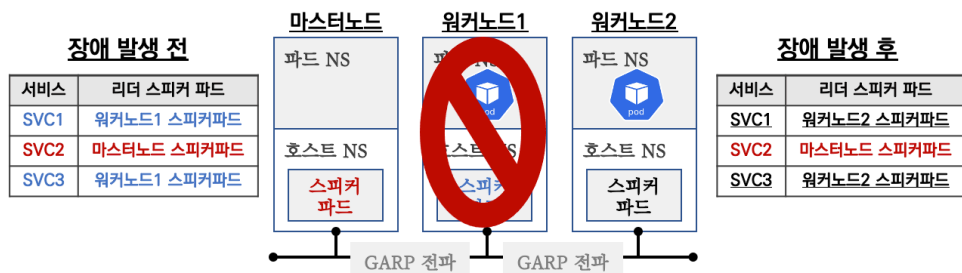
# 이후 체인 규칙은 동일한 내용으로 생략
```

정리하자면 MetalLB Layer2 모드는 ExternalIP 에 대한 소유자가 있는 노드가 서비스 트래픽을 처리하며, 해당 노드에 IPTABLES 규칙에 의해서 각 노드에 있는 파드로 랜덤 부하분산 접속 됩니다.

7.2.6 Failover 테스트

장애 발생 <Failover> 시 동작 과정을 알아보겠습니다.

[그림 7-11] 리더 스피커파드 장애 발생 시



위 그림 처럼 장애 발생 전 워커노드1의 스피커파드가 SVC1, SVC3 의 리더 스피커 파드 역할을 하고 있는 상태에서, 워커노드1에 장애가 발생하게 되면 남아있는 스피

커퍼드들은 워커노드1의 장애를 인지하게 됩니다.

이후 장애가 발생한 스피커퍼드가 소유한 서비스의 ExternalIP 에 대한 리더 파드를 다시 선출하게 됩니다. 예를 들면 워커노드2가 SVC1, SVC3에 대한 리더 스피커퍼드가 선출이 되게 되면, 해당 파드는 GARP 로 자신이 SVC1, SVC3 ExternalIP에 대한 소유라고 전파를 하게 됩니다.

다만 장애 발생 시 문제가 있다는 인지를 확인하는 시간과 ARP 정보가 전파되는 시간이 필요하여 대략 20초~1분 이내의 장애 지속 시간이 발생하게 됩니다.

워커노드 중 1대를 중지하여 장애를 발생시키고, 장애시간을 확인해보겠습니다.

중지하려는 워커노드는 저희 경우 현재 워커노드1이 SVC1, SVC3 의 리더 스피커퍼드가 배포되어 있어서, 워커노드1을 재부팅 하려합니다.

만약 실습을 따라하시다가 워커노드2에 리더 스피커퍼드가 배포되었다면, 워커노드2를 중지하시면 됩니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 router 가상머신에 SSH 접속하여 명령어를 실행합니다.

[터미널3] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널2] ExternalIP 에 대한 ARP 모니터링 : 워커노드1에 SVC1,SVC3 의 리더 스피커퍼드가 배포되어 있음
# watch 툴은 기본 매 2초마다 실행하여 결과를 출력하여, 변동된 값을 확인 할 수 있음
root@router:~# watch -d 'ip neighbor | grep enp0s8 | sort'
192.168.10.10 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
192.168.10.101 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE
192.168.10.102 dev enp0s8 lladdr 08:00:27:94:6a:a0 STALE
192.168.10.201 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE
192.168.10.202 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
192.168.10.203 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE

# [터미널3] curl 로 서비스 ExternalIP 반복 접속 시도 : curl 접속 후 1초 쉬고, 다시 curl 접속을 반복함
while true;do curl -s --connect-timeout 1 --interface 192.168.20.200 $SVC1EXIP |grep name:date;echo:sleep 1;done
```

Hostname: webpod2

Tue May 10 22:11:21 KST 2022

...(생략)...

[터미널1] 마스터노드에서 워커노드1(혹은 워커노드2)에 명령어 실행

```
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w1 hostname  
k8s-w1
```

[터미널1] 마스터노드에서 워커노드1(혹은 워커노드2)를 재부팅 실행 : 혹은 워커노드1에 접속하여 poweroff 실행

```
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w1 poweroff
```

[터미널3] curl 로 서비스 ExternalIP 반복 접속 시도 : 결과 확인

저의 경우에는 반복해서 curl 정상 접속 출력이 나올때까지 45초 정도 장애 시간이 발생하였음

```
while true;do curl -s --connect-timeout 1 --interface 192.168.20.200 $SVC1EXIP |grep name:date;echo:sleep 1;done
```

[터미널2] ExternalIP 에 대한 ARP 모니터링 : 결과 확인

워커노드2에 배포된 스피커파드가 SVC1, SVC3 의 리더 스피커파드가 되었음을 확인

```
root@router:~# watch -d 'ip neighbor | grep enp0s8 | sort'
```

```
192.168.10.10 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
```

```
192.168.10.101 dev enp0s8 lladdr 08:00:27:37:7a:bf STALE
```

```
192.168.10.102 dev enp0s8 lladdr 08:00:27:94:6a:a0 STALE
```

```
192.168.10.201 dev enp0s8 lladdr 08:00:27:94:6a:a0 STALE
```

```
192.168.10.202 dev enp0s8 lladdr 08:00:27:f0:8e:03 STALE
```

```
192.168.10.203 dev enp0s8 lladdr 08:00:27:94:6a:a0 STALE
```

MetalLB Layer2 모드인 경우 노드 장애나 MetalLB 파드에 장애가 발생 시 10초~1분 정도의 장애 지속 시간이 소요됩니다.

만약 실무에서 MetalLB Layer2 모드 사용 시, 해당 장애 지속 시간이 운영하고 있는 애플리케이션 동작에 문제가 없는 지 꼭 사전 장애 관련 테스트를 진행하셔서, 최적화 및 장애 대응 방안을 수립하시기를 권장합니다.

참고로 MetalLB Layer2 모드의 LoadBalancer 서비스에서 ExternalTrafficPolicy: local 설정을 사용 시 리더 스피커파드가 존재하는 노드에 배치된 파드만 접속되며, 다른 노드에 있는 파드는 전달을 하지 못합니다. 만약 리더 스피커파드가 선정된 노

드에 파드가 없다면 접속 불가 상태로 문제가 발생합니다.

정리하자면 MetalLB Layer2 모드에서 ExternalTrafficPolicy: local 설정은 하지 말 것을 권장합니다.

그럼 실습 환경을 중간 정리하겠습니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널4] 은 해당 실습 PC의 터미널 창에서 명령어를 실행합니다.

```
터미널
# [터미널4] 현재 전원 OFF 되어 있는 워커노드1(혹은 노드2)를 다시 전원 ON을 실행
vagrant up k8s-w1 (혹은 vagrant up k8s-w2)

# [터미널1] 마스터노드에서 다음 실습을 위해 리소스를 삭제
root@k8s-m:~# kubectl delete pod,svc --all
root@k8s-m:~# kubectl delete -n metallb-system configmaps config
```

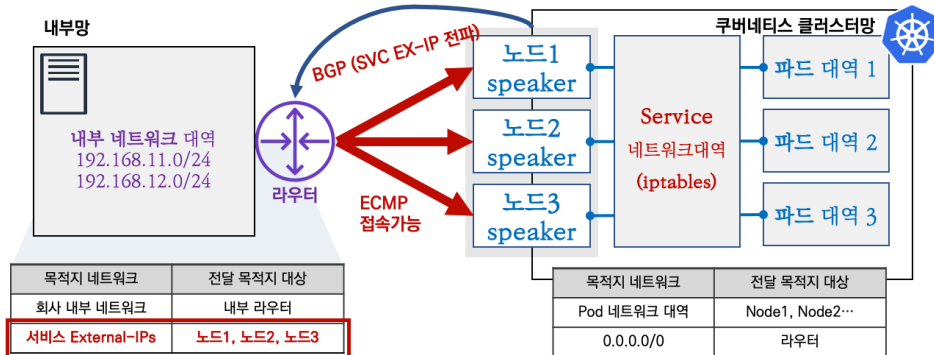
7.3 MetalLB BGP 모드

MetalLB BGP 모드 동작을 알아보겠습니다.

7.3.1 BGP 모드 소개

MetalLB에 BGP 모드 설정 시, 스피커파드는 BGP 프로토콜을 통해 LoadBalancer 서비스의 'External IP' 전파합니다. 스피커파드와 연동을 위해 BGP가 동작하는 라우터 <Router> 네트워크 장비가 필요합니다. 보통 네트워크 팀에서 관리하는 장비이다 보니, 쿠버네티스를 운영 관리하는 플랫폼팀과 업무 협조를 통해서 작업을 진행하게 됩니다.

[그림 7-12] MetalLB BGP 모드 동작



라우터 장비에 MetalLB 스피커파드와 연동 할 수 있게 설정을 한 후에는 LoadBalancer 서비스의 'External IP'가 자동으로 라우터 장비의 라우팅 정보에 업데이트가 됩니다.

이후 사내의 내부망에서 내부 직원은 LoadBalancer 서비스의 'External IP'로 접속 시 라우터 장비를 통해서 쿠버네티스 클러스터망으로 접속하게 됩니다.

만약 서비스 리소스를 삭제 시, 자동으로 External IP 라우팅 정보도 삭제 됩니다.

7.3.2 MetalLB BGP 모드 설정

7.2.2 에서 MetalLB 를 설치 하였다면, MetalLB 설치 는 안하셔도 됩니다.

다만, 7장 실습 환경의 쿠버네티스를 배포하고, 바로 7.3 장의 BGP 모드 실습 진행을 위해서는 마스터노드에서 MetalLB 설치를 진행하셔야 됩니다.

```
터미널
# SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m

# MetalLB 관련 파드가 사용할 네임스페이스 생성
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/namespace.yaml

# MetalLB 설치
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/metallb.yaml
```

7.2 장에 이어서 실습하는 경우에는 아래 부터 진행하시면 됩니다.

그럼 MetalLB BGP 모드 설정을 진행하겠습니다.

```
터미널
# MetalLB BGP 모드를 위한 컨피그맵 확인
root@k8s-m:~# cat ~/book-k8s-network/7/4-metallb-bgp.yaml
...(생략)...
data:
  config: |
    peers:
      - peer-address: 192.168.10.254 # 일반적으로 BGP 가 동작하는 라우터 장비의 IP이며, 저해 실습 환경은 router 가상머신임
        peer-asn: 64512 # BGP AS 번호이며, 사설 AS 번호 중 하나를 사용
        my-asn: 64512
    address-pools:
      - name: default
        protocol: bgp # MetalLB 에 BGP 모드를 설정
        avoid-buggy-ips: true # 예를 들면 172.20.1.0, 172.20.1.255 IP 주소를 ExternalIP 에 할당하지 않는 설정
        addresses:
          - 172.20.1.0/24 # LoadBalancer 서비스 생성 시 사용하는 ExternalIP 주소 대역
```

```
# MetalLB BGP 모드를 위한 컨피그맵 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/4-metallb-bgp.yaml
```

7.3.3 파드 & 서비스 생성

마스터노드에서 7.2.3 과 7.2.4 에서 생성한 파드와 서비스를 다시 생성하겠습니다.

```
터미널
# 파드 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/2-webpod.yaml

# 파드 생성 확인 : 파드의 IP는 실습 환경에 따라 다를 수 있음
root@k8s-m:~# calicoctl get wep

WORKLOAD  NODE    NETWORKS      INTERFACE
webpod1   k8s-w1  172.16.158.2/32  cali778ba511bf9
webpod2   k8s-w2  172.16.184.2/32  cali798bceab6f5

# LoadBalancer 타입의 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/3-svc1b.yaml

# LoadBalancer 타입의 서비스 생성 확인 : EXTERNAL-IP가 서비스 마다 할당되며, 실습 환경에 따라 다를 수 있음
root@k8s-m:~# kubectl get svc

NAME        TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)        AGE
svc1        LoadBalancer  10.200.1.49    172.20.1.1   80:30683/TCP    35s
svc2        LoadBalancer  10.200.1.207   172.20.1.2   80:32192/TCP    35s
svc3        LoadBalancer  10.200.1.46    172.20.1.3   80:31377/TCP    35s
...(생략)...
```

router 가상머신에서 라우팅 정보를 확인해보겠습니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 router 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널2] SSH 접속 : router 가상머신 접속
vagrant ssh router
```

```
# [터미널2] FRR 라우팅 설정 확인 : MetalLB 와 연동하기 위한 BGP 설정이 적용되어 있음
root@router:~# vtysh -c 'show running' | grep 'router bgp' -A12
...(생략)...

# [터미널2] FRR 라우팅 정보 확인 : 서비스의 ExternalIP 를 마스터/워커1/워커2 노드로 라우팅 전달 정보 확인
root@router:~# vtysh -c 'show ip route bgp'
...(생략)...

B)* 172.20.1.1/32 [200/0] via 192.168.10.10, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.101, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.102, enp0s8, weight 1, 00:09:49
B)* 172.20.1.2/32 [200/0] via 192.168.10.10, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.101, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.102, enp0s8, weight 1, 00:09:49
B)* 172.20.1.3/32 [200/0] via 192.168.10.10, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.101, enp0s8, weight 1, 00:09:49
*
    via 192.168.10.102, enp0s8, weight 1, 00:09:49#

# [터미널2] ip route 명령어로 라우팅 정보 확인
root@router:~# ip -c route | grep 172.20 -A3
...(생략)...
```

쿠버네티스 LoadBalancer 서비스의 ExternalIP 를 BGP 프로토콜을 통해서 router 가상머신에 전파되는 것을 확인하였습니다. 참고로 아래 그림은 BGP 패킷의 내용이며, 172.20.1.5 ExternalIP 를 BGP 로 전파하게 됩니다.

[그림 7-13] BGP 패킷 정보 (예시)

```
> Frame 27: 114 bytes on wire (912 bits), 114 bytes captured (912 bits)
> Ethernet II, Src: 08:00:27:76:e3:77, Dst: 08:00:27:84:d8:82
> Internet Protocol Version 4, Src: 192.168.100.102, Dst: 192.168.100.50
> Transmission Control Protocol, Src Port: 55143, Dst Port: 179, Seq: 87,
  > Border Gateway Protocol - UPDATE Message
    Marker: ffffffffffffffffffffffffffffffffff
    Length: 48
    Type: UPDATE Message (2)
    Withdrawn Routes Length: 0
    Total Path Attribute Length: 20
    > Path attributes
      > Path Attribute - ORIGIN: INCOMPLETE
      > Path Attribute - AS_PATH: 64511
      > Path Attribute - NEXT_HOP: 192.168.100.102
    > Network Layer Reachability Information (NLRI)
      > 172.20.1.5/32
        NLRI prefix length: 32
        NLRI prefix: 172.20.1.5
```

7.3.4 서비스 접속

client 가상머신에서 서비스에 접속해보겠습니다.

7.2.4 서비스 접속에 트래픽 전달 과정과 동일하게 client 는 router 가상머신을 통해서 라우팅 처리되어 각 노드로 전달되어 통신이 이루어지게 됩니다.

[터미널3] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널3] SSH 접속 : client 가상머신 접속
vagrant ssh client

# [터미널3] 172.20.1.0 대역 라우팅 정보 확인 : 해당 대역은 router 가상머신에 전달됨을 확인
root@client:~# ip -c route | grep 172.20.1
172.20.1.0/24 via 192.168.20.254 dev enp0s8 proto static

# [터미널3] 서비스 ExternalIP 를 변수에 지정
root@client:~# SVC1EXIP=$(kubectl get svc svc1 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@client:~# SVC2EXIP=$(kubectl get svc svc2 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')
root@client:~# SVC3EXIP=$(kubectl get svc svc3 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

# [터미널3] curl 로 서비스 ExternalIP 접속 시도 : 정상적으로 파드에 접속됨을 확인
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC2EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC3EXIP | grep Hostname
Hostname: webpod2

# [터미널3] for 문을 이용하여 서비스1 ExternalIP 로 100번 접속을 시도 : 파드 2개 부하분산 접속 확인
for i in {1..100}; do curl -s --interface 192.168.20.200 $SVC1EXIP | grep Hostname; done | sort | uniq -c | sort -nr
53 Hostname: webpod1
47 Hostname: webpod2

# [터미널3] curl 접속 확인 : RemoteAddr 주소는 웹 파드에 인입 패킷의 출발지 주소(IP, Port)가 출력됨
# 접속한 client 의 IP는 172.20.1.1 이지만, 파드에 인입되는 출발지IP는 172.16.158.0 으로 확인됨
```

```
# 즉, NodePort 기본 동작과 동일하게 인입한 노드의 인터페이스로 SNAT 되어서 최종 파드로 전달됨
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | egrep 'Hostname|RemoteAddr|Host:'
Hostname: webpod2
RemoteAddr: 172.16.158.0:42623
Host: 172.20.1.1
```

IPTABLES 규칙 역시 7.2.5 서비스 접속에서 알아본 IPTABLES 와 동일합니다. 차이점은 KUBE-SERVICES 규칙에 192.168.10.201 IP 대신 172.20.1.1 IP가 지정되어 있습니다. 즉, ExternalIP 의 IP 만 다르고, 나머지 체인 규칙 처리 동작은 동일합니다.

정리하자면 MetalLB BGP 모드는 BGP가 동작하는 라우터 장비를 통해서 ExternalIP 를 쿠버네티스 노드로 전달합니다. 이후 해당 노드에 IPTABLES 규칙에 의해서 각 노드에 있는 파드로 랜덤 부하분산 접속 됩니다.

7.3.5 externalTrafficPolicy 설정

MetalLB Layer2 모드에 externalTrafficPolicy: Local 설정은 권장하지 않지만, MetalLB BGP 모드에 externalTrafficPolicy: Local 설정을 권장합니다.

이유는 설정 시 파드가 있는 노드로만 라우팅이 설정되며, 별도의 추가 부하분산 과정을 수행하지 않습니다. 또한 노드에서 SNAT 처리를 하지 않아서, 클라이언트의 IP를 보존할 수 있습니다.

실습을 통해서 알아보겠습니다.

 터미널

```
# [터미널1] 마스터노드에서 서비스에 externalTrafficPolicy: Local 설정 적용
root@k8s-m:~# kubectl patch svc svc1 -p '{"spec":{"externalTrafficPolicy": "Local"}}'
root@k8s-m:~# kubectl patch svc svc2 -p '{"spec":{"externalTrafficPolicy": "Local"}}'
root@k8s-m:~# kubectl patch svc svc3 -p '{"spec":{"externalTrafficPolicy": "Local"}}'
```

```
# [터미널2] router 가상머신에서 FRR 라우팅 정보 확인

# 현재 파드는 워크노드1/노드2에 배치되어 있으며, 라우팅이 파드가 있는 노드로만 설정되어 있음을 확인
root@router:~# vtysh -c 'show ip route bgp'

...(생략)...

B)* 172.20.1.1/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:39
*
    via 192.168.10.102, enp0s8, weight 1, 00:01:39
B)* 172.20.1.2/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:39
*
    via 192.168.10.102, enp0s8, weight 1, 00:01:39
B)* 172.20.1.3/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:39
*
    via 192.168.10.102, enp0s8, weight 1, 00:01:39

# [터미널3] client 가상머신에서 curl 접속 확인

# RemoteAddr 에 client 의 IP 가 보존되어 인입되었음을 확인
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | egrep 'Hostname|RemoteAddr|Host:'

Hostname: webpod1

RemoteAddr: 192.168.20.200:41672

Host: 172.20.1.1
```

실습에서 확인 한 것처럼, 서비스에 연동된 파드가 없는 노드로는 라우팅 전달이 되지 않습니다. 이를 통해 최적을 경로 라우팅이 가능해집니다.

또한 클라이언트의 출발지 IP를 파드에서 수집할 수 있습니다.

추가로 워크노드2에 배포된 webpod2 를 삭제하고 라우팅을 확인해보겠습니다.

```
터미널

# [터미널1] webpod2 파드 삭제

root@k8s-m:~# kubectl delete pod webpod2

pod "webpod2" deleted

# [터미널1] 파드 정보 확인 : 현재는 서비스에 연동된 파드는 워크노드1에만 존재

root@k8s-m:~# kubectl get pod -owide

NAME      READY  STATUS   RESTARTS  AGE   IP           NODE      NOMINATED NODE  READINESS GATES
webpod1   1/1    Running  0          69m   172.16.158.2 k8s-w1    <none>           <none>

# [터미널2] router 가상머신에서 FRR 라우팅 정보 확인 : 이제 워크노드1 쪽으로만 라우팅 처리함
root@router:~# vtysh -c 'show ip route bgp'
```

```

...(생략)...
B)* 172.20.1.1/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:57
B)* 172.20.1.2/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:57
B)* 172.20.1.3/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:57

# [터미널3] curl 로 서비스 ExternalIP 접속 시도 : 정상적으로 파드에 접속됨을 확인
root@client:~# curl -s --interface 192.168.20.200 $SVC1EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC2EXIP | grep Hostname
Hostname: webpod1
root@client:~# curl -s --interface 192.168.20.200 $SVC3EXIP | grep Hostname
Hostname: webpod1

# [터미널1] 다음 실습을 위해서 다시 webpod2 를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/2-webpod.yaml

```

현재 실습 환경의 router 리눅스는 externalTrafficPolicy: Local 설정 시 한쪽의 노드로만 접속이 되지만, 실무 환경의 라우터 장비에서는 ECMP 라우팅의 옵션을 통해서 노드에 분산 접속을 할 수 있습니다.

7.3.6 Failover 테스트

MetalLB BGP 모드의 환경에서 장애 발생<Failover> 시 동작 과정을 알아보겠습니다.

파드가 있는 워커노드에 장애가 발생 시 라우터 장비는 BGP 프로토콜에 의해서 장애 상황을 인지하고 해당 워커노드로의 라우팅 정보를 바로 삭제합니다. BGP 에 BFD<Bidirectional Forwarding Detection> 를 사용 시 대부분 1초 이내에 상대 장비의 장애를 알 수 있습니다. 아쉽게도 MetalLB 기본 BGP 모드는 BFD를 지원하지 않지만, MetalLB FRR(베타 기능) BGP 모드는 BFD 를 지원합니다.

결론적으로 MetalLB **BGP 모드** 환경에서 장애 발생 시 거의 무중단의 연속성 있는 애플리케이션 서비스가 가능합니다.

그럼 저희 실습 환경에서 테스트를 해보겠습니다.

워커노드 중 1대를 중지하여 장애를 발생시키고, 장애시간을 확인해보겠습니다.

```
터미널

# [터미널2] router 가상머신에서 FRR 라우팅 정보를 지속적으로 확인
root@router:~# watch -d 'vtysh -c 'show ip route bgp''
...(생략)...
B)* 172.20.1.1/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:12:31
*
   via 192.168.10.102, enp0s8, weight 1, 00:12:31
B)* 172.20.1.2/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:12:31
*
   via 192.168.10.102, enp0s8, weight 1, 00:12:31
B)* 172.20.1.3/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:12:31
*
   via 192.168.10.102, enp0s8, weight 1, 00:12:31

# [터미널3] curl 로 서비스 ExternalIP 반복 접속 시도 : curl 접속 후 1초 쉬고, 다시 curl 접속을 반복함
# 현재 워커노드2에 배포된 webpod2 로 접속 중임을 확인
while true;do curl -s --connect-timeout 1 --interface 192.168.20.200 $SVC3EXIP |grep name;date;echo;sleep 1;done
Hostname: webpod2
Tue May 10 22:11:21 KST 2022
...(생략)...

# [터미널1] 마스터노드에서 워커노드2(혹은 워커노드1)를 재부팅 실행 : 혹은 워커노드2에 접속하여 poweroff 실행
# 저의 경우 바로 위 curl 접속 결과 webpod2 에 접속되었으므로 워커노드2를 재부팅 실행함
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w2 poweroff

# [터미널3] curl 로 서비스 ExternalIP 반복 접속 시도 : 결과 확인
# 저의 경우에는 무중단으로 연속성 있게 파드에 접속되었음
while true;do curl -s --connect-timeout 1 --interface 192.168.20.200 $SVC1EXIP |grep name;date;echo;sleep 1;done
Hostname: webpod2
Wed May 11 03:28:50 KST 2022

Hostname: webpod1
Wed May 11 03:28:51 KST 2022
```



```
# [터미널2] router 가상머신에서 FRR 라우팅 정보를 지속적으로 확인 : 결과 확인
```

```
# 워크노드2가 중지되었으므로, 워크노드1로만 라우팅 설정되어 있음을 확인
```

```
root@router:~# watch -d 'vtysh -c 'show ip route bgp''
```

```
B)* 172.20.1.1/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:40
```

```
B)* 172.20.1.2/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:40
```

```
B)* 172.20.1.3/32 [200/0] via 192.168.10.101, enp0s8, weight 1, 00:01:40
```

MetalLB BGP 모드의 경우 노드 장애나 MetalLB 파드에 장애가 발생 시 경우에 따라 1초 이내 혹은 BGP 장애 확인 시간만 소요가 되면 바로 정상 서비스가 됩니다. 즉, MetalLB 를 사용하고 계시다면, MetalLB Layer2 모드 보다는 BGP 모드 사용을 적극 권장합니다. 다만 BGP 설정 및 라우팅 최적화가 필요하여 네트워크 팀과 협조를 하시길 추천드립니다.

추가로 파드가 특정 노드에 집중되어 있을 경우 트래픽 집중이 될 수 있으므로, 파드를 노드별로 균등하게 배치하는 것도 권장합니다.

이상으로 MetalLB 관련 이론과 실습을 마무리 하였습니다.

그럼 다음 DNS 실습 환경을 위해서 중간 정리하겠습니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널4] 은 해당 실습 PC의 터미널 창에서 명령어를 실행합니다.

```
터미널
```

```
# [터미널4] 현재 전원 OFF 되어 있는 워크노드1(혹은 노드2)를 다시 전원 ON을 실행
```

```
vagrant up k8s-w1 (혹은 vagrant up k8s-w2)
```

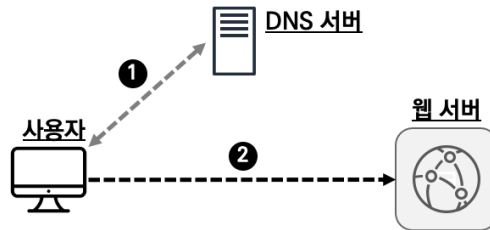
```
# [터미널1] 마스터노드에서 다음 실습을 위해 리소스를 삭제
```

```
root@k8s-m:~# kubectl delete pod,svc --all
```

7.4 CoreDNS

일반적으로 DNS<Domain Name System> 은 도메인 주소를 IP 주소로 변환해주는 역할을 합니다. DNS 를 통해 사용자나 애플리케이션은 IP 주소 대신 도메인 주소로 접속이 가능합니다.

[그림 7-14] 도메인 주소를 통한 통신 흐름



(1) 예를 들면 사용자가 blog.cloudneta.net 도메인 주소로 접속을 시도하게 되면 먼저 자신의 운영체제에 설정된 DNS 서버에 해당 도메인 주소에 매핑된 IP 주소를 알기 위해서 DNS 서버에 질의(Query) 를 하게 됩니다.

DNS 서버는 해당 도메인 주소에 매핑된 IP 주소 정보를 사용자에게 반환합니다.

(2) 사용자는 IP 주소로 웹 서버와 통신을 진행합니다.

마스터노드에서 DNS 질의 과정을 간단히 확인해보겠습니다.

```
터미널
# 현재 설정되어 있는 DNS 서버 정보 확인
root@k8s-m:~# cat /etc/resolv.conf
nameserver 1.1.1.1 # 네임서버(Local DNS 서버) 1.1.1.1 을 우선해서 질의함
nameserver 127.0.0.53
options edns0 trust-ad

# nslookup 툴로 blog.cloudneta.net 도메인 주소에 대한 IP 확인을 위한 도메인 주소 질의
root@k8s-m:~# nslookup blog.cloudneta.net
Server:      1.1.1.1    # 1.1.1.1 주소에 도메인주소 질의를 수행함을 의미
Address:     1.1.1.1#53
```

Non-authoritative answer:

Name: blog.cloudneta.net

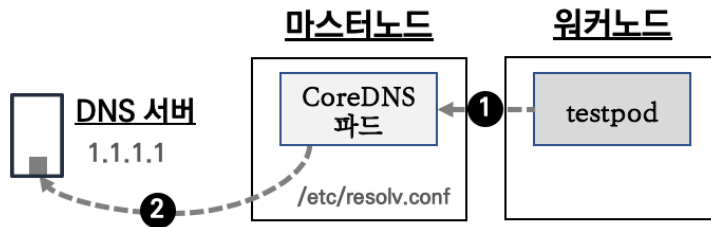
Address: 52.219.60.137 # 해당 도메인 주소에 대한 IP 주소는 52.219.60.137 이며, 해당 정보를 DNS서버가 응답해줌

7.4.1 CoreDNS 소개

CoreDNS 는 쿠버네티스 클러스터 내에서 DNS 서버 역할을 하며, 서비스 디스커버리 <Service Discovery> 기능을 제공합니다.

CoreDNS 는 파드와 서비스를 위한 DNS 레코드 <Record> 를 생성합니다. 이를 통해 파드와 서비스에 IP 이외에 도메인 주소로도 접속이 가능합니다.

[그림 7-15] CoreDNS 를 통한 도메인 질의



CoreDNS 는 디플로이먼트로 배포가 되며, 기본적으로 2개^(Replicas=2) 의 파드가 배포됩니다.

[그림 7-15] 에서 (1)번의 클러스터 내부 도메인 주소 패턴을 도메인 질의 시 과정을 설명드리겠습니다.

CoreDNS 는 쿠버네티스 클러스터를 위한 내부 도메인 주소 패턴을 제공합니다.

첫 번째는 서비스에서 사용하며 '서비스이름.네임스페이스이름.svc.cluster.local' 입니다. 예를 들면 네임스페이스는 test 에 속해 있는 서비스 이름이 svc1 일 경우, 'svc1.test.svc.cluster.local' 도메인 주소로 접속이 가능합니다.

두 번째는 **파드**에서 사용하며 '파드-IP-주소.네임스페이스이름.pod.cluster.local' 입니다. 예를 들면 네임스페이스는 dev 에 속해 있는 파드 pod1(10.1.1.1) 일 경우, '10-1-1-1.dev.pod.cluster.local' 도메인 주소로 접속이 가능합니다. 이 경우 파드 이름이 아니라 파드의 IP 에서 "." 대신 "-" 을 사용해야 됩니다.

[그림 7-15] 에서 (2)번의 외부 도메인 주소를 질의과정을 설명드리겠습니다.

쿠버네티스 클러스터 내부에서 파드나 서비스 도메인 주소가 아닌 인터넷 상에 외부 도메인 주소에 접속을 위한 도메인 질의는 어떻게 처리가 될까요?

CoreDNS 기본 설정은 자신이 관리하는 클러스터 도메인 주소가 아닐 경우 호스트에 있는 /etc/resolv.conf 에 설정을 통해서 질의를 할 수 있게 전달<Forward> 합니다. 예를 들면, 쿠버네티스 내부의 파드에서 kubernetes.io 접속을 위해 도메인 질의를 하게 되면 CoreDNS 는 자신이 관리하는 클러스터 도메인 주소가 아니므로, 호스트에 있는 /etc/resolv.conf 에 설정되어 있는 (IP는 예시) nameserver 1.1.1.1 로 질의를 하게 됩니다.

마스터노드에서 CoreDNS 를 확인해보겠습니다.

```

터미널
# CoreDNS 디플로이먼트 정보 확인
root@k8s-m:~# kubectl get deploy -n kube-system coredns

NAME      READY   UP-TO-DATE   AVAILABLE   AGE
coredns   2/2     2             2           134m

# CoreDNS 파드 정보 확인
root@k8s-m:~# kubectl get pod -n kube-system -l k8s-app=kube-dns

NAME                                READY   STATUS    RESTARTS   AGE
coredns-64897985d-7q9jv             1/1     Running   0          135m
coredns-64897985d-jmc2q             1/1     Running   0          135m

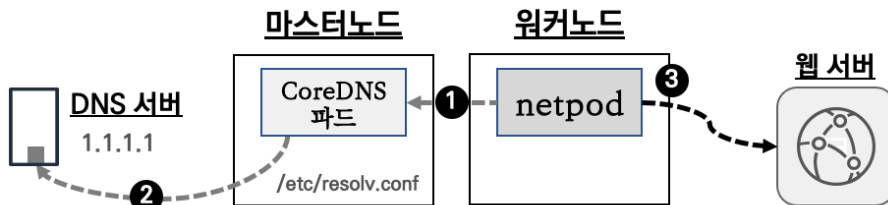
# CoreDNS 설정 확인
root@k8s-m:~# kubectl describe configmaps -n kube-system coredns
...(생략)...
```

```
forward . /etc/resolv.conf { # 클러스터 도메인이 아닌 경우 도메인 질의를 전달 방법 지정
...(생략)...
```

7.4.2 파드 DNS

파드를 생성하여 파드에서 외부 도메인 접속 시 도메인 질의 과정을 알아보자.

[그림 7-16] 파드에서 CoreDNS 를 통한 도메인 질의



- (1) 파드는 CoreDNS 에 도메인 질의를 요청하고, CoreDNS 는 자신이 관리하는 클러스터 도메인(기본값 cluster.local) 이 아닐 경우, (2) 해당 호스트에 설정을 참고하여 외부 도메인 <DNS> 서버에 질의를 하게 됩니다. 이후 CoreDNS 는 외부 도메인 서버로 부터 응답을 받고 다시 파드에게 도메인 질의 결과를 전달합니다.
- (3) 파드는 응답 받은 IP 정보를 확인하고, 해당 IP(웹 서버) 로 접속을 합니다.

마스터노드에서 실습을 통해 알아보겠습니다.

```
터미널
# 테스트 환경을 단순화 하기 위해서 CoreDNS 파드의 갯수를 2개에서 1개로 줄임
root@k8s-m:~# kubectl scale deployment -n kube-system coredns --replicas=1

# CoreDNS 파드 정보 확인 : 1개 배포되어 있음을 확인
root@k8s-m:~# kubectl get pod -n kube-system -l k8s-app=kube-dns
NAME                                READY  STATUS   RESTARTS  AGE
coredns-64897985d-4jb79             1/1    Running  0          27m

# netpod 이름의 테스트용 파드를 배포
cat book-k8s-network/6/1-testpod.yaml | sed -e 's/testpod/netpod/g' | kubectl apply -f -
```

```
# 파드 정보 확인
root@k8s-m:~# kubectl get pod

NAME      READY   STATUS    RESTARTS   AGE
netpod    1/1     Running   0           2m20s

# netpod 에서 zsh 쉘 실행
root@k8s-m:~# kubectl exec -it netpod -- zsh

-----

...(생략)...

Welcome to Netshoot! (github.com/nicolaka/netshoot)
netpod>

# netpod 에서 네임서버 정보 확인
netpod> cat /etc/resolv.conf
nameserver 10.200.1.10 # 네임서버 IP 확인

search default.svc.cluster.local svc.cluster.local cluster.local # 검색 기본 도메인 (3개) 설정 확인
options ndots:5 # 도메인 질의 하기 전 검색 기본 도메인을 적용 시 도메인 계층 "." 최소 갯수

# netpod 에서 kubernetes.io 도메인 질의 확인
# ndots:5 옵션 설정으로 검색 기본 도메인 3개를 kubernetes.io 뒤에 자동으로 붙여서 먼저 반복 질의를 수행함
# 결국 검색 기본 도메인 질의에는 해당 정보가 없으니, 맨 마지막에 kubernetes.io 도메인은 질의에 응답을 받음
# 맨 마지막 kubernetes.io 도메인 질의는 CoreDNS 파드에 있는 /etc/resolv.conf 설정을 참고하여 질의함
netpod> nslookup -type=A kubernetes.io -debug | grep QUESTIONS -A1

QUESTIONS:
    kubernetes.io.default.svc.cluster.local, type = A, class = IN
--
QUESTIONS:
    kubernetes.io.svc.cluster.local, type = A, class = IN
--
QUESTIONS:
    kubernetes.io.cluster.local, type = A, class = IN
--
QUESTIONS:
    kubernetes.io, type = A, class = IN

# netpod 에서 kubernetes.io. (도메인 끝에 . 점 dot 있음) 절대 (Absolute) 도메인 질의 확인
# 절대 도메인 이름의 맨 끝에 . 점의 의미는 루트를 나타내며, 뒤에 생략된 도메인 계층이 없다는 것을 의미함
netpod> nslookup -type=A kubernetes.io. -debug | grep QUESTIONS -A1
```

```
QUESTIONS:
    kubernetes.io, type = A, class = IN

# netpod 에서 zsh 쉘 빠져나오기
netpod> exit
-----

# CoreDNS 파드에 연동된 서비스 정보 확인 : 파드에서 바라보는 네임서버의 IP를 확인
root@k8s-m:~# kubectl get svc -n kube-system kube-dns
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kube-dns  ClusterIP  10.200.1.10   <none>        53/UDP,53/TCP,9153/TCP  152m

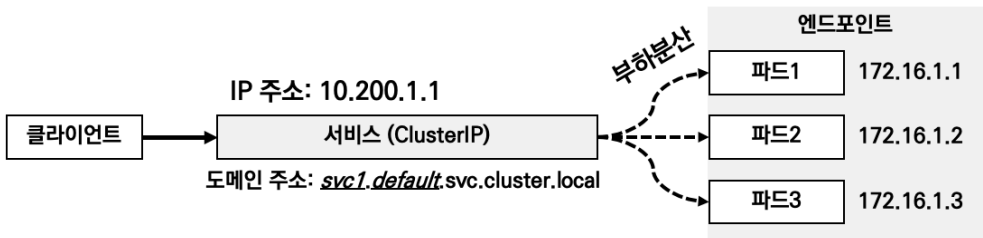
# kube-dns 서비스의 엔드포인트 정보 확인 : CoreDNS 파드의 IP 정보를 확인
root@k8s-m:~# kubectl describe svc -n kube-system kube-dns | grep Endpoints
Endpoints:          172.16.116.1:53
...(생략)...
```

CoreDNS 파드가 배포된 노드에서 CoreDNS 파드와 연동된 호스트의 veth 인터페이스로 tcpdump 를 실행 시, 파드에서 도메인 질의하는 패킷을 확인 할 수 있습니다.

7.4.3 서비스 DNS

서비스를 생성하여 파드에서 서비스 도메인 접속 시 도메인 질의 과정을 알아보자.

[그림 7-17] CoreDNS 를 통한 서비스 도메인 질의



서비스(ClusterIP)를 생성 시, 서비스 IP 이외에도 서비스 도메인(A Record)이 생성됩니다.
참고로 A(IPv4) 레코드는 도메인 주소를 IP 주소로 변환 시 사용되는 레코드입니다.

예를 들면 default 네임스페이스에서 svc1 이라는 서비스를 생성하게 되면,
'svc1.default.cluster.local' 이라는 서비스 도메인이 생성됩니다.

서비스 도메인 주소 패턴은 '〈서비스명〉.〈네임스페이스명〉.〈쿠버네티스 클러스터 도메인〉' 입니다. '쿠버네티스 클러스터 도메인'은 주로 기본값인 'cluster.local'를 사용하지만, 변경 설정이 가능합니다.

그럼 6장에서 사용했던 디플로이먼트와 서비스를 배포하여 확인해보겠습니다.

```
터미널
# 디플로이먼트를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/4-deploy.yaml

# 파드 생성 확인
root@k8s-m:~# kubectl get pod -l app=deploy-websrv
NAME                                READY   STATUS    RESTARTS   AGE
deploy-echo-d4cc6b75d-4rj2s        1/1     Running   0           74s
deploy-echo-d4cc6b75d-gz2cf        1/1     Running   0           74s
deploy-echo-d4cc6b75d-pc2fg        1/1     Running   0           74s

# NodePort 서비스 생성 : NodePort 는 ClusterIP 를 포함하고 있음
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/5-svc2.yaml

# NodePort 서비스 생성 확인
root@k8s-m:~# kubectl get svc svc2
NAME    TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
svc2    NodePort    10.200.1.42    <none>         9000:32357/TCP   2m13s

# netpod 에서 zsh 쉘 실행
root@k8s-m:~# kubectl exec -it netpod -- zsh
-----

# netpod 에서 svc2 서비스 도메인 주소로 질의 확인 : 아래 4가지 도메인 질의 결과는 동일함
```



```

netpod> nslookup svc2
netpod> nslookup svc2.default
netpod> nslookup svc2.default.svc
netpod> nslookup svc2.default.svc.cluster.local
...(생략)
Name:      svc2.default.svc.cluster.local
Address: 10.200.1.42

# netpod 에서 svc2 서비스 도메인 주소로 curl 접속 확인 : 정상 접속 확인
netpod> curl -s svc2:9000 | grep Hostname
netpod> curl -s svc2.default:9000 | grep Hostname
netpod> curl -s svc2.default.svc:9000 | grep Hostname
netpod> curl -s svc2.default.svc.cluster.local:9000 | grep Hostname
Hostname: deploy-echo-d4cc6b75d-4rj2s

# netpod 에서 zsh 쉘 빠져나오기
netpod> exit
-----

# 실습 완료 후 디플로이먼트와 서비스 삭제
root@k8s-m:~# kubectl delete svc svc2
root@k8s-m:~# kubectl delete deploy deploy-echo

```

7.4.4 ExternalName 서비스

서비스 유형의 마지막인 ExternalName 을 알아보도록 하겠습니다.

ExternalName 서비스 유형은 지금까지 학습한 서비스 유형과 동작 방향이 다릅니다.

기존의 ClusterIP, NodePort, LoadBalancer 서비스의 경우 해당 IP 로 통신 접속을 주로 하지만, ExternalName 서비스 유형은 CoreDNS 에 **CNAME 레코드**를 생성 하여, 외부 시스템에 연동 시에 주로 사용하게 됩니다. 참고로 CNAME 레코드는 A 레코드의 IP 매핑과 달리 별칭 이름, 즉 도메인 주소를 매핑합니다.

[그림 7-18] ExternalName 서비스 타입을 활용



예를 들면 파드가 외부에 있는 google.com^(예시) 도메인 주소에 접속을 시도 할 경우, google.com 주소가 주기적으로 변경 될 경우, 변경 된 주소로 접속을 해야합니다. 하지만 ExternalName 서비스를 사용 시에는, 쿠버네티스 클러스터 내부에서 고정 된 도메인 주소로 접속할 수 있습니다.

쿠버네티스 운영자가 ExternalName 서비스를 생성하여, (1) 쿠버네티스 내부의 사용자 혹은 애플리케이션은 고정적인 exname 도메인으로 접속을 하면 됩니다. (2) 이때 CoreDNS 는 exname 도메인의 응답을 운영자가 설정해둔 CNAME, 즉 실제 접속해야 되는 도메인 주소를 알려주게 됩니다.

실습을 통해 알아보겠습니다.

```

터미널
# ExternalName 서비스를 위한 yaml 파일 확인
root@k8s-m:~# cat ~/book-k8s-network/7/5-svc-exname.yaml
apiVersion: v1
kind: Service
metadata:
  name: exname      # 쿠버네티스 내부에서 접속을 시도할 도메인 주소
spec:
  type: ExternalName # 서비스 유형이 ExternalName임을 확인
  externalName: google.com # CNAME에 매핑된 실제 접속할 도메인 주소

# ExternalName 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/5-svc-exname.yaml

```

```
# netpod 파드에서 exanme 도메인 주소 질의
root@k8s-m:~# kubectl exec -it netpod -- nslookup exname
...(생략)...
exname.default.svc.cluster.local      canonical name = google.com.  # CNAME 레코드 매핑 정보 확인
Name:      google.com
Address: 172.217.31.174 # google.com 도메인 질의 결과 IP 응답 확인
...(생략)...

# netpod 파드에서 exanme 도메인 주소로 curl 접속 시도 : IP는 위 google.com 도메인 질의 결과와 동일
root@k8s-m:~# kubectl exec -it netpod -- curl -s -v exname
* Trying 172.217.31.174:80...
* Connected to exname (172.217.31.174) port 80 (#0)
> GET / HTTP/1.1
> Host: exname
> User-Agent: curl/7.82.0
> Accept: */*
...(생략)...
```

이상으로 서비스의 4가지 유형에 대해서 알아보았습니다.

다음은 서비스의 4가지 유형으로 분류는 되지 않지만, **스테이트풀셋** <StatefulSets> 에서 파드에 접속 시 활용되는 **헤드리스 서비스** <Headless Service> 에 대해서 알아보겠습니다.

7.4.5 스테이트풀셋 & 헤드리스 서비스

스테이트풀셋 <StatefulSets> 은 **상태 저장** <Stateful> 을 갖는 파드를 관리하는 리소스입니다.

애플리케이션에 따라 **상태 저장** <Stateful> 이 필요한 경우가 있고 상태 저장이 필요하지 않는 경우가 있습니다. 대표적으로 **데이터베이스**의 경우 상태 저장이 반드시 필요합니다. 예를 들면 데이터베이스를 파드로 배포 시, 해당 파드가 삭제 후 재생성 되었을 때 저장된 데이터가 삭제되고 초기화 상태가 되면 문제가 발생합니다.

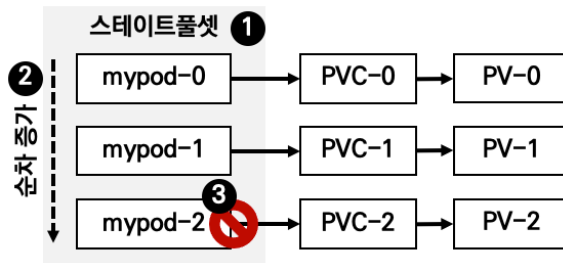
[그림 7-19] ExternalName 서비스 타입을 활용



쿠버네티스에서도 일반적으로 **파드**는 상태 저장이 필요하지 않는 애플리케이션을 말합니다. 이런 경우 보통 '**가축**'으로 비유합니다. 가축의 경우 고유한 이름을 지정하지 않고, 특별하지 않으니 언제든지 삭제되고 생성 될 수 있습니다.

이와 반대로 상태 저장이 필요한 파드는 '**애완동물**'로 비유할 수 있습니다. 애완동물의 경우에 고유한 이름을 부여하여 다른 애완동물과 구별 할 수 있습니다. 이와 같이 상태 저장이 필요한 파드는 고유한 **이름**과 **저장소**를 가질 수 있습니다. 이를 통해 상태 저장이 필요한 애플리케이션을 쿠버네티스 환경에서도 운영 할 수 있습니다. 이렇게 상태 저장이 필요한 파드를 관리하는 리소스를 **스테이트풀셋**이라고 합니다.

[그림 7-20] 스테이트풀셋 동작



(1) 스테이트풀셋으로 배포된 파드의 이름은 '<이름>-N' 이며, N 은 0 부터 순차적으로 증가합니다. 위 [그림 7-20] 처럼 첫 번째 파드는 mypod-0 이고, 두 번째 파드는 mypod-1, 세 번째 파드는 mypod-2 입니다.

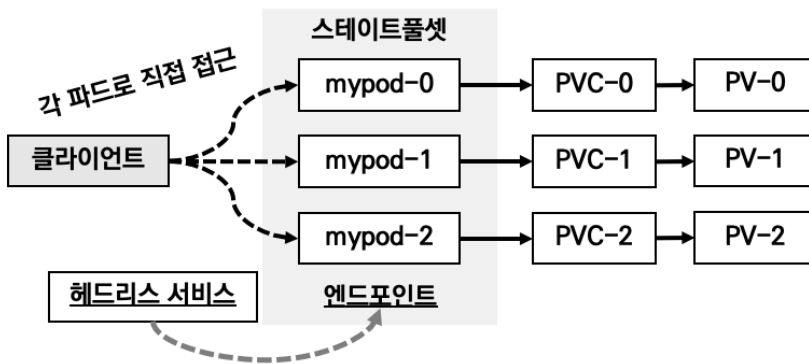
(2) 스테이트풀셋에 레플리카<Replicas> 가 3일 경우 3개의 파드가 동시에 생성 되지 않고, mypod-0 이 생성 완료 후 mypod-1 이 생성되고, 그 다음 mypod-2 가 생

성됩니다. 이렇게 스테이트풀셋 파드는 순차적으로 생성됩니다. 반대로 파드가 삭제될 경우에는 역순으로 삭제됩니다.

(3) 만약 강제로 mypod-2 파드를 삭제하게 되면 다시 재생성이 됩니다. 이때 기존에 사용하던 PVC/PV 가 연결이 되어서 저장소를 유지할 수 있습니다.

참고로 PV<PersistentVolume> 는 영구 볼륨 자체를 말합니다. 쉽게 저장소, 저장공간으로 이해하시면 됩니다. PVC<PersistentVolumeClaim> 는 저장소 사용을 위한 요청입니다.

[그림 7-21] 헤드리스 서비스를 통한 파드 접속



스테이트풀셋에서 헤드리스<Headless> 서비스를 연동할 경우 각 개별 파드로의 도메인 주소로 직접 접속이 가능합니다.

헤드리스 서비스는 ClusterIP 가 생성되지 않습니다. 헤드리스 서비스의 도메인 주소에 질의를 하게 되면, 엔드포인트들의 파드의 IP 주소를 응답 받게 됩니다. 클라이언트는 헤드리스 서비스의 도메인 주소로 접속이 가능합니다.

또한 헤드리스 서비스는 개별 파드로도 직접 접근이 가능합니다. 각 개별 파드에 고유한 파드 도메인 접속 주소가 생성되어 해당 주소를 통해 파드에 접속이 가능합니다.

실습을 통해 알아보겠습니다.

터미널

```
# 스테이트풀셋과 헤드리스 서비스 배포를 위한 yaml 파일 확인

root@k8s-m:~# cat ~/book-k8s-network/7/6-svc-headless.yaml

apiVersion: apps/v1

kind: StatefulSet # 스테이트풀셋 리소스를 배포

metadata:
  name: sts-echo
  ... (생략) ...
  ---

apiVersion: v1

kind: Service

metadata:
  name: sts-echo

spec:
  type: ClusterIP
  clusterIP: None # 헤드리스 서비스는 ClusterIP 가 생성되지 않음

  ports:
    - name: webport
      port: 9000
      targetPort: 8080 # 헤드리스 서비스는 ClusterIP 가 없으므로, 파드로 직접 접속하기 위해 targetPort 를 사용

# 스테이트풀셋과 헤드리스 서비스 배포 : 파드가 순차적으로 생성됨을 확인

root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/6-svc-headless.yaml && kubectl get pod -w
```

NAME	READY	STATUS	RESTARTS	AGE
netpod	1/1	Running	0	41m
sts-echo-0	0/1	ContainerCreating	0	0s
sts-echo-0	0/1	ContainerCreating	0	1s
sts-echo-0	1/1	Running	0	2s
sts-echo-1	0/1	Pending	0	0s
sts-echo-1	0/1	Pending	0	0s
sts-echo-1	0/1	ContainerCreating	0	0s
sts-echo-1	0/1	ContainerCreating	0	0s
sts-echo-1	1/1	Running	0	1s
sts-echo-2	0/1	Pending	0	0s
sts-echo-2	0/1	Pending	0	0s
sts-echo-2	0/1	ContainerCreating	0	0s
sts-echo-2	0/1	ContainerCreating	0	1s
sts-echo-2	1/1	Running	0	2s

스테이트풀셋과 파드 정보 확인 : 파드이름 명 뒤에 순차적인 숫자로 고유한 파드명으로 배포됨을 확인

```
root@k8s-m:~# kubectl get sts,pod
```

NAME	READY	AGE
statefulset.apps/sts-echo	3/3	53s

NAME	READY	STATUS	RESTARTS	AGE
pod/netpod	1/1	Running	0	42m
pod/sts-echo-0	1/1	Running	0	53s
pod/sts-echo-1	1/1	Running	0	51s
pod/sts-echo-2	1/1	Running	0	50s

헤드리스 서비스 정보 확인 : ClusterIP 가 없음을 확인

```
root@k8s-m:~# kubectl get svc sts-echo
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
sts-echo	ClusterIP	None	<none>	9000/TCP	3m59s

헤드리스 서비스에 엔드포인트 정보 확인

```
root@k8s-m:~# kubectl get endpoints sts-echo
```

NAME	ENDPOINTS	AGE
sts-echo	172.16.158.7:8080,172.16.184.10:8080,172.16.184.11:8080	5m7s

netpod 에서 zsh 쉘 실행

```
root@k8s-m:~# kubectl exec -it netpod -- zsh
```

netpod 에서 sts-echo 서비스 도메인 주소로 질의 확인 : 엔드포인트의 파드 IP 리스트를 확인

```
netpod> nslookup sts-echo
```

...(생략)...

Name: sts-echo.default.svc.cluster.local

Address: 172.16.184.11

Name: sts-echo.default.svc.cluster.local

Address: 172.16.184.10

Name: sts-echo.default.svc.cluster.local

Address: 172.16.158.7

for 문을 이용하여 sts-echo 서비스 주소로 100번 접속을 시도 : 파드 3개 부하분산 접속 확인

```
netpod> for i in {1..100}; do curl -s sts-echo:8080 | grep Hostname; done | sort | uniq -c | sort -nr
```

39 Hostname: sts-echo-1

32 Hostname: sts-echo-0

29 Hostname: sts-echo-2

```

# sts-echo 도메인 주소에 SRV 레코드 타입으로 질의 확인
# SRV (Service Record) 레코드는 서비스의 위치를 나타내며, 호스트네임과 포트번호로 구성됨
# 아래 출력 내용 중 8080 은 포트번호이며, sts-echo-0/1/2..(생략).. 은 각 파드의 고유한 도메인 주소임
netpod> nslookup -type=SRV sts-echo
...(생략)...
sts-echo.default.svc.cluster.local    service = 0 33 8080 sts-echo-0.sts-echo.default.svc.cluster.local.
sts-echo.default.svc.cluster.local    service = 0 33 8080 sts-echo-1.sts-echo.default.svc.cluster.local.
sts-echo.default.svc.cluster.local    service = 0 33 8080 sts-echo-2.sts-echo.default.svc.cluster.local.

# 첫 번째 파드 도메인 주소 질의 확인 : 172.16.184.10 은 sts-echo-0 파드의 IP 임을 확인
netpod> nslookup -type=A sts-echo-0.sts-echo.default.svc.cluster.local
...(생략)...
Name:      sts-echo-0.sts-echo.default.svc.cluster.local
Address: 172.16.184.10

# 두 번째 파드 도메인 주소 질의 확인 : 172.16.158.7 은 sts-echo-1 파드의 IP 임을 확인
netpod> nslookup -type=A sts-echo-1.sts-echo.default.svc.cluster.local
...(생략)...
Name:      sts-echo-1.sts-echo.default.svc.cluster.local
Address: 172.16.158.7

# 세 번째 파드 도메인 주소 질의 확인 : 172.16.184.11 은 sts-echo-2 파드의 IP 임을 확인
netpod> nslookup -type=A sts-echo-2.sts-echo.default.svc.cluster.local
...(생략)...
Name:      sts-echo-2.sts-echo.default.svc.cluster.local
Address: 172.16.184.11

# 첫 번째 파드 도메인 주소로 curl 접속 확인 : 첫 번째 파드로 직접 접속됨을 확인
netpod> curl -s sts-echo-0.sts-echo:8080 | grep Hostname
Hostname: sts-echo-0

# for 문을 이용하여 첫 번째 파드 도메인 주소로 100번 접속을 시도 : 첫 번째 파드로만 접속 됨을 확인
for i in {1..100}; do curl -s sts-echo-0.sts-echo:8080 | grep Hostname; done | sort | uniq -c | sort -nr
100 Hostname: sts-echo-0

# netpod 에서 zsh 쉘 빠져나오기
netpod> exit

```



```
-----  
  
# 실습 완료 후 실습 자원 삭제  
root@k8s-m:~# kubect! delete sts,svc sts-echo  
  
# 실습 PC 에서 vagrant 로 가상 머신 삭제  
exit  
vagrant destroy -f
```

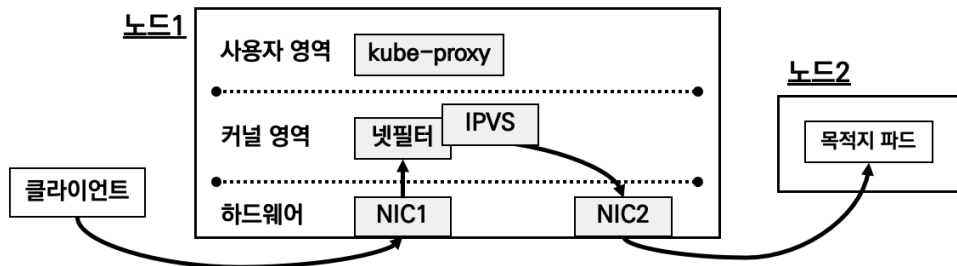
스테이트풀셋과 헤드리스 서비스를 알아보았습니다. 스테이트풀셋을 사용하고 특정 파드에 직접 접속이 필요한 경우에 헤드리스 서비스를 통해서 가능합니다.

7.5 IPVS Proxy 모드

지금까지 iptables proxy 모드 환경에서 서비스 동작을 알아보았습니다.

이번에는 IPVS proxy 모드를 알아보겠습니다.

[그림 7-22] IPVS proxy 모드



IPVS proxy 모드는 3가지 모드 중 가장 성능 효율적인 모드입니다.

IPVS<IP Virtual Server> 는 넷필터에서 동작하는 Layer 4 로드밸런서<Load Balancer> 입니다.

iptables 보다 좀 더 높은 성능 처리를 보여주고, 규칙 갯수를 줄일 수 있습니다.

또한 다양한 **부하분산 알고리즘**을 제공합니다.

라운드 로빈<Round Robin> 스케줄은 우선순위를 두지 않고 요청을 대상 목적지 파드로 돌아가면서 전달합니다.

최소 연결<Least Connection> 스케줄은 목적지 파드의 연결 갯수가 가장 적은 곳을 우선해서 전달합니다.

목적지 해싱<Destination Hashing> 스케줄은 목적지 IP 주소로 해시값을 계산하여 목적지 파드를 결정하여 전달합니다.

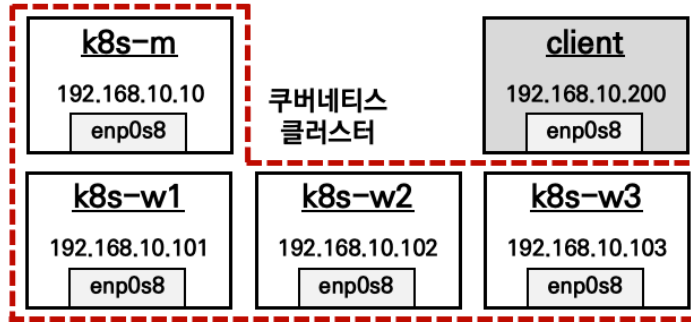
출발지 해싱<Source Hashing> 스케줄은 출발지 IP 주소로 해시값을 계산하여 목적지 파드를 결정하여 전달합니다.

이외에도 응답 속도가 가장 빠른 목적지 파드를 선택하는 **최단 지연**<Shortest Expected Delay> 스케줄 등이 있습니다.

7.5.1 실습 환경 구성

칼리코 CNI 를 사용하며, 칼리코의 Direct 모드로 파드 간 통신을 합니다.

[그림 7-23] 실습 환경 구성도



vagrant 로 5대의 가상머신을 배포하며, 쿠버네티스 클러스터에 마스터 노드 1대와, 워커 노드 3대(k8s-w1 k8s-w2 k8s-w3) 로 구성됩니다. client 이름의 가상머신은 서비스 접속 테스트 용도로 사용됩니다.

실습 환경은 6장과 동일하며 차이점은 iptables proxy 모드가 아니라 ipvs proxy 모드로 설정이 되어 배포가 완료됩니다.

실습 환경을 배포하겠습니다.

```
터미널
# 베이그런트 스크립트 파일을 다운로드
curl -O https://raw.githubusercontent.com/gasida/book-k8s-network/main/7-5/Vagrantfile

# 배포 실행 : PC 환경에 따라 다르지만, 보통 20분~30분 정도 소요
vagrant up

# 배포된 가상머신 상태 확인 : 5대의 가상머신이 정상 동작 중
vagrant status

Current machine states:
k8s-m                running (virtualbox)
k8s-w1              running (virtualbox)
k8s-w2              running (virtualbox)
k8s-w3              running (virtualbox)
```

client running (virtualbox)

```
# SSH 접속 : 각각 가상머신 접속
vagrant ssh k8s-m
vagrant ssh k8s-w1
vagrant ssh k8s-w2
vagrant ssh k8s-w3
vagrant ssh client
```

6장에서 설치한 쿠버네티스 스크립트와 다른 부분들을 살펴보겠습니다.

터미널

```
# SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m

# 마스터 노드를 설정하는 스크립트 파일 확인
cat master.sh
...(생략)...
cat <(EOT) $HOME/install.yaml        # kubeadm 초기화 설정을 파일로 작성
apiVersion: kubeadm.k8s.io/v1beta3
kind: InitConfiguration
...(생략)...
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
networking:
  dnsDomain: mylab.local              # 클러스터 도메인 설정
  serviceSubnet: 10.200.1.0/24
  podSubnet: 172.16.0.0/16
---
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: "ipvs"                          # ipvs proxy 모드 설정
ipvs:
  strictARP: true                      # ipvs 모드에서 MetalLB 설치 시 반드시 필요한 옵션으로 ARP 요청 시 동작 관련 설정
EOT
kubeadm init --config $HOME/install.yaml    # kubeadm 초기화를 파일을 통하여 실행
...(생략)...
```

설치 후 환경을 간단히 확인해보겠습니다.

```
터미널

# kube-proxy 모드 확인 : ipvs 모드 정보 확인

root@k8s-m:~# kubectl get cm -n kube-system kube-proxy -o yaml | egrep 'mode|strictARP'

    strictARP: true
    mode: ipvs

# 마스터 노드에 kube-ipvs0 네트워크 인터페이스 정보 확인 : 해당 인터페이스에 여러개의 IP가 할당

root@k8s-m:~# ip -d -c addr show kube-ipvs0

5: kube-ipvs0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group default
    link/ether 72:58:46:2f:20:76 brd ff:ff:ff:ff:ff:ff promiscuity 0 minmtu 0 maxmtu 0
    dummy numrxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535
    inet 10.200.1.1/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
    inet 10.200.1.10/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
    inet 10.200.1.91/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever

# 모든 네임스페이스에 서비스 정보 확인 : 위 IP는 서비스에 할당된 IP 임을 확인

root@k8s-m:~# kubectl get svc -A

NAMESPACE   NAME           TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
default     kubernetes     ClusterIP   10.200.1.1   <none>        443/TCP          55m
kube-system kube-dns        ClusterIP   10.200.1.10  <none>        53/UDP,53/TCP,9153/TCP 55m
kube-system metrics-server  ClusterIP   10.200.1.91  <none>        443/TCP          55m

# ipvsadm 톨로 부하분산 되는 정보 확인 : 서비스의 IP와 서비스에 연동되어 있는 파드의 IP 를 확인

root@k8s-m:~# ipvsadm -Ln

IP Virtual Server version 1.2.1 (size=4096)

Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port           Forward Weight ActiveConn InActConn
TCP  10.200.1.1:443 rr
  -> 192.168.10.10:6443            Masq   1    4    0
TCP  10.200.1.10:53 rr
  -> 172.16.158.1:53               Masq   1    0    0
  -> 172.16.158.2:53               Masq   1    0    0
...(생략)...
```

```
# 간편하게 동작 확인을 위해서 마스터노드에서 다른 워커노드에 리모트로 명령 수행 확인
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w1 hostname
k8s-w1
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w2 hostname
k8s-w2
root@k8s-m:~# sshpass -p qwe123 ssh -o StrictHostKeyChecking=no root@k8s-w3 hostname
k8s-w3

# 마스터노드에서 각 워커노드에 kube-ipvs0 네트워크 인터페이스 정보 확인 : 모든 노드에서 동일함
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w1 ip -d -c addr show kube-ipvs0
5: kube-ipvs0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group default
    link/ether e6:41:be:a5:65:25 brd ff:ff:ff:ff:ff:ff promiscuity 0 minmtu 0 maxmtu 0
    dummy numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535
    inet 10.200.1.1/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
    inet 10.200.1.10/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever
    inet 10.200.1.91/32 scope global kube-ipvs0
        valid_lft forever preferred_lft forever

root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w2 ip -br -c -4 addr show kube-ipvs0
kube-ipvs0      DOWN      10.200.1.1/32 10.200.1.10/32 10.200.1.91/32
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w3 ip -d -c addr show kube-ipvs0
kube-ipvs0      DOWN      10.200.1.1/32 10.200.1.10/32 10.200.1.91/32
```

6장의 실습에서 사용했던 파드와 서비스를 배포하여 IPVS 동작에 대해서 알아보겠습니다.

7.5.2 ClusterIP 서비스

ipvs proxy 모드에서 ClusterIP 서비스 동작에 대해서 알아보겠습니다.

ClusterIP 서비스 생성 시 kube-proxy 에 의해서 모든 노드에 iptables 규칙과 ipvs 부하분산이 설정됩니다.

마스터 노드에 testpod 파드를 배포하고, 워커노드에 webpod 파드를 배포 한 후 ClusterIP 서비스를 생성합니다.

터미널

```
# 마스터노드에 testpod 를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/1-testpod.yaml

# 워커노드에 webpod 를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/2-webpod.yaml

# ClusterIP 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/3-svc1.yaml

# 파드 정보 확인
root@k8s-m:~# kubectl get pod -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
testpod	1/1	Running	0	90s	172.16.116.0	k8s-m	<none>	<none>
webpod1	1/1	Running	0	60s	172.16.158.3	k8s-w1	<none>	<none>
webpod2	1/1	Running	0	60s	172.16.184.0	k8s-w2	<none>	<none>
webpod3	1/1	Running	0	60s	172.16.24.0	k8s-w3	<none>	<none>

```
# ClusterIP 서비스 확인
root@k8s-m:~# kubectl get svc svc1
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
svc1	ClusterIP	10.200.1.216	<none>	9000/TCP	65s

```
# 모든 노드에 kube-ipvs0 인터페이스 정보 확인
# ClusterIP 생성 시 kube-ipvs0 인터페이스에 ClusterIP 가 할당되는 것을 확인
root@k8s-m:~# ip -br -c -4 addr show kube-ipvs0
kube-ipvs0      DOWN          10.200.1.1/32 10.200.1.10/32 10.200.1.91/32 10.200.1.216/32
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w1 ip -br -c -4 addr show kube-ipvs0
kube-ipvs0      DOWN          10.200.1.1/32 10.200.1.10/32 10.200.1.91/32 10.200.1.216/32
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w2 ip -br -c -4 addr show kube-ipvs0
kube-ipvs0      DOWN          10.200.1.1/32 10.200.1.10/32 10.200.1.91/32 10.200.1.216/32
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w3 ip -br -c -4 addr show kube-ipvs0
kube-ipvs0      DOWN          10.200.1.1/32 10.200.1.10/32 10.200.1.91/32 10.200.1.216/32

# ClusterIP 와 Port 를 변수에 지정
```

```

root@k8s-m:~# CIP=$(kubectl get svc svc1 -o jsonpath={.spec.clusterIP})
root@k8s-m:~# CPORT=$(kubectl get svc svc1 -o jsonpath={.spec.ports[0].port})

# ipvsadm 톨로 부하분산 되는 정보 확인
# 10.200.1.216(TCP 9000) 인입 시 3곳의 목적지로 라운드로빈(rr)로 부하분산하여 전달됨을 확인
# 3곳의 목적지는 각각 서비스에 연동된 목적지 파드 3개이며, 전달 시 출발지 IP는 마스크레이딩 변환 처리
root@k8s-m:~# ipvsadm -Ln -t $CIP:$CPORT

Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port          Forward Weight ActiveConn InActConn
TCP  10.200.1.216:9000 rr
  -> 172.16.24.0:80               Masq    1      0      0
  -> 172.16.158.3:80              Masq    1      0      0
  -> 172.16.184.0:80              Masq    1      0      0

# ipvsadm 톨로 부하분산 되는 현재 연결 정보 확인 : 추가로 --rate 도 있음
root@k8s-m:~# ipvsadm -Ln -t $CIP:$CPORT --stats

Prot LocalAddress:Port          Conns   InPkts  OutPkts  InBytes OutBytes
  -> RemoteAddress:Port
TCP  10.200.1.216:9000           0        0        0        0        0
  -> 172.16.24.0:80              0        0        0        0        0
  -> 172.16.158.3:80             0        0        0        0        0
  -> 172.16.184.0:80             0        0        0        0        0

# 워크노드1에 ipvsadm 톨로 부하분산 정보 확인 : 마스터노드와 동일함을 확인
root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w1 ipvsadm -Ln -t $CIP:$CPORT

Prot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port          Forward Weight ActiveConn InActConn
TCP  10.200.1.216:9000 rr
  -> 172.16.24.0:80               Masq    1      0      0
  -> 172.16.158.3:80              Masq    1      0      0
  -> 172.16.184.0:80              Masq    1      0      0

# 마스터노드에서 ipvsadm 모니터링 실행 : ClusterIP 접속 시 아래 처럼 연결 정보 확인됨
root@k8s-m:~# watch -d ipvsadm -Ln -t $CIP:$CPORT --stats

Prot LocalAddress:Port          Conns   InPkts  OutPkts  InBytes OutBytes
  -> RemoteAddress:Port
TCP  10.200.1.216:9000           1        6        4      401      494
  -> 172.16.24.0:80              1        6        4      401      494
  -> 172.16.158.3:80             0        0        0        0        0

```



```
-> 172.16.184.0:80      0      0      0      0      0
```

```
# [터미널2] 신규 터미널에서 마스터노드에 SSH 접속 후 testpod 에서 ClusterIP 로 접속 확인
```

```
root@k8s-m:~# kubectl exec testpod -it -- curl -s $CIP:$CPORT | grep Hostname
```

```
Hostname: webpod3
```

```
# [터미널2] testpod 에서 ClusterIP 로 100번 접속 확인 : 라운드로빈(rr) 스케줄에 의해 균등하게 부하분산됨
```

```
for i in {1..100}; do kubectl exec testpod -it -- curl -s $CIP:$CPORT | grep Hostname; done | sort | uniq -c | sort -nr
```

```
34 Hostname: webpod2
```

```
33 Hostname: webpod3
```

```
33 Hostname: webpod1
```

```
# [터미널2] iptables 규칙 확인 : ipset list 를 활용
```

```
root@k8s-m:~# iptables -t nat -S | grep KUBE-CLUSTER-IP
```

```
-A KUBE-SERVICES ! -s 172.16.0.0/16 -m comment --comment "Kubernetes service cluster ip + port for masquerade purpose" -m set --match-set KUBE-CLUSTER-IP dst,dst -j KUBE-MARK-MASQ
```

```
-A KUBE-SERVICES -m set --match-set KUBE-CLUSTER-IP dst,dst -j ACCEPT
```

```
# [터미널2] ipset list 정보를 확인 : KUBE-CLUSTER-IP 이름은 아래 6개의 IP:Port 조합을 지칭
```

```
# 예를 들면 ipset list 를 사용하지 않을 경우 6개의 iptables 규칙이 필요하지만, ipset 사용 시 1개의 규칙으로 가능
```

```
root@k8s-m:~# ipset list KUBE-CLUSTER-IP
```

```
Name: KUBE-CLUSTER-IP
```

```
Type: hash:ip,port
```

```
Revision: 6
```

```
Header: family inet hashsize 1024 maxelem 65536 bucketsize 12 initval 0x7f15fbac
```

```
Size in memory: 488
```

```
References: 2
```

```
Number of entries: 6
```

```
Members:
```

```
10.200.1.10,tcp:53
```

```
10.200.1.1,tcp:443
```

```
10.200.1.10,udp:53
```

```
10.200.1.10,tcp:9153
```

```
10.200.1.91,tcp:443
```

```
10.200.1.216,tcp:9000
```

```
# 다음 실습을 위해 리소스 삭제
```

```
root@k8s-m:~# kubectl delete pod,svc --all
```

7.5.3 NodePort 서비스

ipvs proxy 모드에서 NodePort 서비스 동작에 대해서 알아보겠습니다.

워크노드에 디플로이먼트를 생성하고, 클라이언트 가상머신을 통해서 NodePort 접속을 수행합니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

```
터미널
# [터미널1] SSH 접속 : 마스터 노드 접속
vagrant ssh k8s-m

# [터미널2] SSH 접속 : client 가상머신 접속
vagrant ssh client

# [터미널1] 디플로이먼트를 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/4-deploy.yaml

# [터미널1] NodePort 서비스 생성
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/5-svc2.yaml

# [터미널1] 파드 생성 확인
root@k8s-m:~# kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
deploy-echo-d4cc6b75d-7sfrx         1/1     Running   0           59s
deploy-echo-d4cc6b75d-bmhlq         1/1     Running   0           59s
deploy-echo-d4cc6b75d-pzc26         1/1     Running   0           59s

# [터미널1] 서비스 생성 확인
root@k8s-m:~# kubectl get svc svc2
NAME    TYPE       CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
svc2    NodePort   10.200.1.143 <none>        9000:30445/TCP   69s

# [터미널2] NodePort 변수에 지정
root@client:~# NPORT=$(kubectl get service svc2 -o jsonpath='{.spec.ports[0].nodePort}')
```

```
# [터미널2] client에서 마스터:NodePort 로 접속 시도
root@client:~# curl -s 192.168.10.10:$NPORT | egrep 'Host|client|host'
Hostname: deploy-echo-d4cc6b75d-7d8tg # 목적지 파드 이름
      client_address=192.168.10.10 # 목적지 파드에 인입되는 출발지 IP, 즉 클라이언트의 IP 주소
      host=192.168.10.10:30286 # client 가 처음 접속을 시도하는 목적지 정보

# [터미널2] for 문을 이용하여 SVC2 IP 로 100번 접속을 시도 : 파드 3개 부하분산 접속 확인
for i in {1..100}; do curl -s 192.168.10.10:$NPORT | grep Host: done | sort | uniq -c | sort -nr
      34 Hostname: deploy-echo-d4cc6b75d-lwdqc
      34 Hostname: deploy-echo-d4cc6b75d-7d8tg
      32 Hostname: deploy-echo-d4cc6b75d-nmmzp

# [터미널1] NodePort 변수에 지정
root@k8s-m:~# NPORT=$(kubectl get service svc2 -o jsonpath='{.spec.ports[0].nodePort}')

# [터미널1] 마스터노드에서 ipvsadm 모니터링 실행 : NodePort 인입 시 각 파드로 부하분산됨을 확인
root@k8s-m:~# watch -d ipvsadm -Ln -t 192.168.10.10:$NPORT --stats
Prot LocalAddress:Port          Conns   InPkts   OutPkts   InBytes OutBytes
-> RemoteAddress:Port
TCP  192.168.10.10:30445          301     2107     1505     136955  295582
-> 172.16.24.1:8080              101      707      505     45955   99182
-> 172.16.158.4:8080             100      700      500     45500   98200
-> 172.16.184.1:8080             100      700      500     45500   98200

# [터미널1] 다음 실습을 위해 리소스 삭제
root@k8s-m:~# kubectl delete deploy,svc --all
```

7.5.4 LoadBalancer 서비스

ipvs proxy 모드에서 LoadBalancer 서비스 동작에 대해서 알아보겠습니다.

MetalLB 설치 후 Layer 2 모드에서 동작을 확인해보겠습니다.

[터미널1] 은 마스터 노드에 SSH 접속하여 명령어를 실행합니다.

[터미널2] 은 client 가상머신에 SSH 접속하여 명령어를 실행합니다.

터미널

[터미널1] MetalLB 관련 파드가 사용할 네임스페이스를 생성

```
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/namespace.yaml
```

[터미널1] MetalLB 설치

```
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.12.1/manifests/metallb.yaml
```

[터미널1] MetalLB Layer 2 모드를 위한 컨피그맵 생성

```
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/1-metallb-layer2.yaml
```

[터미널1] MetalLB 관련 파드 설치 확인

```
root@k8s-m:~# kubectl get pod -n metallb-system
```

NAME	READY	STATUS	RESTARTS	AGE
controller-57fd9c5bb-phx9v	1/1	Running	0	89s
speaker-5mzrb	1/1	Running	0	89s
speaker-gtw6z	1/1	Running	0	89s
speaker-zsxqh	1/1	Running	0	89s

[터미널1] 파드 생성

```
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/6/2-webpod.yaml
```

[터미널1] LoadBalancer 타입의 서비스 생성

```
root@k8s-m:~# kubectl apply -f ~/book-k8s-network/7/3-svc1b.yaml
```

[터미널1] 파드 생성 확인

```
root@k8s-m:~# kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
webpod1	1/1	Running	0	62s
webpod2	1/1	Running	0	62s
webpod3	1/1	Running	0	62s

[터미널1] 서비스 생성 확인

```
root@k8s-m:~# kubectl get svc,ep
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.200.1.1	<none>	443/TCP	23m
service/svc1	LoadBalancer	10.200.1.60	192.168.10.201	80:30944/TCP	79s
service/svc2	LoadBalancer	10.200.1.236	192.168.10.202	80:32070/TCP	79s
service/svc3	LoadBalancer	10.200.1.186	192.168.10.203	80:30324/TCP	79s

NAME	ENDPOINTS	AGE
endpoints/kubernetes	192.168.10.10:6443	23m
endpoints/svc1	172.16.158.5:80,172.16.184.3:80,172.16.24.2:80	79s
endpoints/svc2	172.16.158.5:80,172.16.184.3:80,172.16.24.2:80	79s
endpoints/svc3	172.16.158.5:80,172.16.184.3:80,172.16.24.2:80	79s

[터미널1] 마스터노드에 kube-ipvs0 인터페이스 정보 확인

LoadBalancer 서비스 당 External-IP 와 Cluster-IP가 할당되어 6개의 IP가 추가 할당되었음을 확인

root@k8s-m:~# ip -c -4 addr show kube-ipvs0

5: kube-ipvs0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN group default

...(생략)...

```

inet 10.200.1.60/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 192.168.10.201/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.200.1.236/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 192.168.10.202/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 10.200.1.186/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever
inet 192.168.10.203/32 scope global kube-ipvs0
    valid_lft forever preferred_lft forever

```

[터미널1] 마스터노드에서 워커노드1의 kube-ipvs0 인터페이스 정보 확인

root@k8s-m:~# sshpass -p qwe123 ssh root@k8s-w1 ip -br -c -4 addr show kube-ipvs0

```

kube-ipvs0    DOWN      10.200.1.10/32 10.200.1.91/32 10.200.1.1/32 10.200.1.60/32 192.168.10.201/32
10.200.1.236/32 192.168.10.202/32 10.200.1.186/32 192.168.10.203/32

```

[터미널2] client 가상머신에서 서비스의 External-IP 변수 지정

root@client:~# SVC1EXIP=\$(kubectl get svc svc1 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

root@client:~# SVC2EXIP=\$(kubectl get svc svc2 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

root@client:~# SVC3EXIP=\$(kubectl get svc svc3 -o jsonpath='{.status.loadBalancer.ingress[0].ip}')

[터미널2] client 가상머신에서 서비스의 External-IP 로 접속 테스트

root@client:~# curl -s \$SVC1EXIP | grep Hostname

Hostname: webpod1

root@client:~#

```

root@client:~# curl -s $SVC2EXIP | grep Hostname
Hostname: webpod3
root@client:~# curl -s $SVC3EXIP | grep Hostname
Hostname: webpod3

# [터미널2] client 가상머신에서 서비스1의 External-IP 로 100번 접속 테스트
root@client:~# for i in {1..100}; do curl -s $SVC1EXIP | grep Hostname; done | sort | uniq -c | sort -nr
    34 Hostname: webpod3
    33 Hostname: webpod2
    33 Hostname: webpod1

# [터미널1] 실험 완료 후 삭제
root@k8s-m:~# kubectl delete pod,svc --all

# 실험 PC 에서 vagrant 로 가상 머신 삭제
exit
vagrant destroy -f

```

이상으로 쿠버네티스 서비스의 모든 유형과 쿠버네티스에서 동작하는 DNS에 대해서 알아보았습니다.

끝